

DP-600 Complete Study Pack

Implementing Analytics Solutions Using Microsoft Fabric — combined reference, tips & tricks, hands-on practice, and full-length assessments.

Contents

1. Power Query / Warehouse Notes	p. 2–3
Visual Query Editor basics, merge & aggregation, SQL query editor, save-as-table, cross-warehouse queries, query folding.	
2. Exam Strategy — Reading Case Studies & Decoding Question Clues	p. 4–5
How DP-600 case studies are built, phrase-to-feature keyword map, elimination tactics, self-check method.	
3. DP-600 Study Guide: Tips & Tricks + Full Answers & Diagrams	p. 6–22
Data Preparation, Semantic Models, Security & Maintenance tips; full walkthroughs of the 12 core data-prep questions; KQL/Sentinel knowledge check.	
4. SQL & DAX Tips and Tricks	p. 23–28
T-SQL execution order, GROUP BY/HAVING, JOIN types, window functions; DAX row/filter context, SUMX vs SUM, CALCULATE, variables, RANKX, USERRELATIONSHIP vs TREATAS, storage modes.	
5. Fabric Security & Governance	p. 29–39
Dynamic data masking, row-level security, column-level security, SQL granular permissions, classification & sensitivity labels, endorsement, OneLake catalog, guided exercises, glossary.	
6. KQL Hands-On Lab Practice	p. 40–55
15 worked KQL exercises against SampleIoTData.Trips — filtering, aggregation, time binning, and render visualizations.	
7. Prepare the Data — 30 MCQ with Answers	p. 56–57
High-level multiple-choice questions covering Get Data, Transform Data, and Query & Analyze Data.	
8. Data Interpretation Practice	p. 58–60
5 scenario questions reading actual result sets, metrics readouts, and diagnostics — KQL output, HAVING-filtered results, Direct Lake fallback, column quality, RLS test-as-role.	
9. Practice Assessment — Attempt 1 (Full Review Report)	p. 61–80
50-question practice exam with every answer, justification, and Microsoft Learn reference, plus a performance summary by skill area and exam objective domain.	
10. Practice Assessment — Attempt 2 (Full Review Report)	p. 81–100
Retake of the 50-question assessment with updated results, progress comparison against Attempt 1, and a refreshed study plan.	

Microsoft Fabric / Synapse Data Warehouse — Power Query Notes

1. Visual Query Editor Basics

- Accessed from the warehouse via **New visual query** (alongside **New SQL query**).
- The **Explorer** pane lists Warehouses → Schemas → Tables (e.g. `DimCity`, `DimCustomer`, `DimDate`, `FactSale`, `FactMovement`, `FactOrder`, `FactPurchase`).
- Dragging two tables into the canvas creates source nodes; a **Merge** step combines them.
- Ribbon actions available in the visual query editor: **Manage columns**, **Reduce rows**, **Sort**, **Transform**, **Combine**, **Save as view**, **View SQL**.
- Output actions: **Download Excel file** and **Visualize results**.

2. Merge Step & Aggregation

- After merging (e.g. `DimCity` + `FactSale`), you can:
 - **Expand** the merged table's columns (e.g. `Expanded FactSale`).
 - **Group rows** to aggregate (e.g. `TotalProfit`, `TotalQuantity` grouped by `CityKey`).
- Once the query is shaped, use **Save as view** to persist the logic as a view, or **Save as table** to materialize results.

3. SQL Query Editor

- **New SQL query** opens a code editor (separate from the visual query editor).
- Supports standard `SELECT` statements against warehouse tables (e.g. querying a `Trip` table with columns like `PickupLatitude`, `DropoffLongitude`, `TripDurationSeconds`, `FareAmount`, `TotalAmount`).
- Bottom tabs: **Data**, **Query**, **Model**.

4. Query Dependencies & Load Settings

- Related/dependent queries (e.g. `Trip` → `Date`) can be viewed via **Highlight related queries** / **Expand related queries**.
- Right-click menu on a query step offers: **Expand**, **Copy**, **Delete**, **Rename**, **Enable load**, **Duplicate**, **Reference**.
- **Enable load** controls whether a query's output is materialized/loaded as a table.

5. Save as Table

- **Save as table** saves query results into a warehouse table (uses `CREATE TABLE AS SELECT` under the hood).
- Steps:
 1. Select the target **warehouse**.
 2. Select a **schema** you have create-table permission on.
 3. Provide a **table name**.
- Once created, the new table appears automatically in the **Explorer**.

6. Cross-Warehouse Queries

- The visual query editor supports combining data **across multiple warehouses**.
- Process: drag tables from different added warehouses onto the canvas, then use a **Merge** activity to join them.
- Example: `store_sales` table from a `sales` warehouse merged with an `item` table from a `marketing` warehouse.
- Multiple warehouses can be added via + **Warehouses** in the Explorer pane.

7. Query Folding & Diagnostics

Query folding = pushing query steps down to execute on the data source instead of locally.

The folding indicator next to each step shows the folding state of the *entire query up to that point* (not just that single step). Diagnostics are **not strictly sequential** — a later step can fold even if an earlier one didn't.

Indicator Types

Indicator	Meaning
Folding	The data source evaluates the query up to this step.
Not folding	Part of the query up to this step is evaluated outside the data source.
Might fold	Folding is decided at runtime (dynamic); common with ODBC/OData connectors.
Opaque	Query plan is inconclusive — often a "constant" table or unsupported connector/transform.
Unknown	No query plan exists (error, or evaluating something other than a table — e.g. list, record, primitive).

Key Behaviors

- A **not-folding** indicator doesn't mean *nothing* folds — it means not *everything* folds. Generally, everything up to the **last folding indicator** still folds; steps after that run locally.
- Example progression:
 - `Source` → inconclusive
 - `Navigation` → folds
 - `Removed columns` → still folds (green line extends)
 - `Capitalize each word` (a transform that never folds) → **stops folding** from this point on
 - `Changed column type` (after a non-folding step) → continues **not folding**
- **Order matters:** if you remove the column that had the non-folding transform applied (e.g. drop the capitalized column), the **optimized query plan can fold again**, even though an earlier step in the applied-steps list didn't fold. Folding depends on both step order and which transformations are applied.

8. Dataflow Gen1 → Dataflow Gen2 (CI/CD) Migration ("Save As")

Prerequisites:

- Fabric capacity (F64 or higher) or Power BI Premium (P) capacity, or trial capacity.
- The tenant/workspace admin must enable Dataflow Gen2.
- Contributor or higher role on the workspace.

Steps to migrate:

1. In the workspace, select ... next to an existing dataflow → **Save as Dataflow Gen2 (CI/CD)**.
 2. In the **Save As** dialog, optionally rename it, then select **Create**.
 3. The new Dataflow Gen2 (CI/CD) opens — review and make any needed changes.
 4. Close the new Dataflow Gen2 (CI/CD) — select **Save and run**.
- This same **Save As** feature also lets you save a Dataflow Gen2 as a new Dataflow Gen2 (CI/CD).
 - **Tip:** For bulk/wide migrations, use the **REST API** instead of doing it dataflow-by-dataflow.
 - **Known limitation:** Scheduled/incremental refresh settings are *not* copied by Save As and must be reconfigured manually; some Dataflow Gen2 capabilities are not yet supported for the CI/CD variant.

Notes compiled from Microsoft Fabric / Synapse Data Warehouse documentation screenshots (visual query editor, query folding diagnostics, and Dataflow Gen2 CI/CD migration).

Reading Case Studies & Decoding Question Clues

A framework for working through DP-600 scenario questions efficiently, plus a keyword-to-concept map for the most common traps.

How DP-600 case studies are built

Case studies front-load a long scenario (Overview → Environment → Requirements) and then ask several independent questions against it. Two things follow from that:

- **You don't need to hold the whole scenario in memory.** Each question usually depends on only one or two requirement bullets. Skim the question stem first, then go find the relevant lines rather than re-reading everything.
- **Requirements are grouped by theme on purpose** (Data Analytics, Data Preparation, Semantic Model, Security, General). A question about permissions almost always maps to the Security Requirements block, not something scattered elsewhere.

The core skill: translate business language into a Fabric feature

Exam writers avoid naming the feature directly — they describe the need, and you're meant to recognize which capability satisfies it. Building an "if you see X, think Y" table is the highest-leverage prep technique.

Phrase pattern in the question	What it's really asking about
"per-minute billing," "dedicated on-demand capacity"	Fabric trial / F-SKU capacity with pause-capable billing, not Premium
"must support branching" for semantic models/reports	Git integration via Azure Repos (the exam's intended answer, not GitHub)
"minimize rows added during refresh" + a column representing creation order	Incremental refresh anchored on the maximum value of that column
"read lakehouse/warehouse data <i>and shortcuts</i> via SQL endpoints"	Grant Read access at the SQL analytics endpoint level, not Viewer on Lakehouse explorer
"read lakehouse data via Lakehouse explorer" (no SQL mention)	A narrower role — Lakehouse-only read via the Files/Tables view
"must be presented as managed tables in Lakehouse explorer"	Data needs to be written as managed Delta tables , not accessed only as shortcuts/external tables
"read via T-SQL or Python" + "semi-structured/unstructured data" + "RLS for T-SQL users"	Points to a Lakehouse , not a Warehouse
"logic maintained in a single location" for a derived rule (e.g. pricing tiers)	A view or computed column in the data store, so every consumer reads one definition
"no access to datasets or reports" but	A narrow item/workspace role — don't over-grant a

"read/write to the data store"	broad role that also exposes reports
"principle of least privilege" appears in the requirements	A flag that the correct answer is the narrowest permission/tool that still satisfies the need
"Direct Lake mode" + RLS defined at the warehouse	Watch for the trap: Direct Lake models querying an RLS-protected source typically fall back to DirectQuery

Elimination tactics for multiple-choice questions

1. **Match the verb to the tool.** "Ingest/load" questions are usually pipelines, dataflows, or notebooks — the deciding clue is often low-code preference (→ dataflow/pipeline) vs. custom transformation logic (→ notebook/Spark).
2. **Distrust answers naming the wrong object type.** If the data lives in a lakehouse, a warehouse-only T-SQL answer (or vice versa) is often a distractor.
3. **Watch for near-duplicate options that swap one word** — minimum vs. maximum, Read vs. Read-Write, Table vs. Tables. These pairs test whether you caught the one qualifying clue in the stem.
4. **For "what should you do first" questions,** look for a dependency chain — a setting usually has to be enabled at a higher scope (tenant → capacity → workspace → item) before a lower-scope setting becomes available.
5. **On HOTSPOT/drag-and-drop items,** read every dropdown option once before picking any — later blanks sometimes clarify earlier ones.

A quick self-check method

For any scenario question, write a one-line summary before looking at the options: "*They need [outcome], constrained by [limitation], for [role].*" If that one-liner doesn't map cleanly onto one answer choice, you've likely missed a qualifying detail in the case study — go back and reread the specific requirement bullet, not the whole case.

Bottom line: DP-600 rewards pattern recognition over memorization. The scenarios repeat the same handful of requirement types (governance, version control, refresh performance, security scoping, ingestion tooling) with different wording each time — once you can spot which pattern a question is testing, the four answer choices usually narrow to one fast.

Scenario clue	Tool
"Orchestrate," "schedule," "copy data," "trigger other activities," move files between systems	Data Factory Pipeline
"Complex transformations," "PySpark," "large-scale," "custom code," "data science"	Notebook (Spark)
Simple ingestion from a database/API on a schedule	Pipeline (Copy activity)

Trick: If the question mentions *both* orchestration AND transformation, the answer is usually "a pipeline that calls a Dataflow Gen2 or Notebook activity" — not one tool alone.

Lakehouse vs. Warehouse

- **Lakehouse** = files + Spark + supports unstructured/semi-structured data + SQL endpoint is *read-only*.
- **Warehouse** = full T-SQL read/write, relational engine, best for traditional BI/SQL developers.
- **Trigger words:** "Data science team needs Spark" → Lakehouse. "SQL developers need full DDL/DML" → Warehouse.

OneLake Shortcuts

- Think "**pointer, not copy.**" Used to reference data in another Lakehouse, ADLS Gen2, S3, etc., without duplicating it.
- **Trigger:** "Without copying the data" / "single source of truth across workspaces" → shortcut.

SQL Analytics Endpoint

- Auto-generated, **read-only** T-SQL view over Lakehouse Delta tables.
- **Trick:** If the question says "write back to the Lakehouse using T-SQL," the SQL endpoint is NOT the answer (it can't write) — that's a Warehouse.

Batch Ingestion

- Pipelines with Copy activity, Dataflow Gen2, or scheduled notebooks. Know the **triggers:** scheduled, tumbling window, event-based.

Query Folding

- **Definition trap:** query folding = pushing Power Query/Dataflow steps back to the source system so the source does the work instead of Fabric.
- **Trigger:** "Improve dataflow performance" / "reduce load on Fabric compute" → check if steps still fold; reorder steps so folding isn't broken (avoid custom columns/renames early in the query).

Pipeline Monitoring & Failures

- Answer is almost always **Monitoring Hub** or the pipeline's **run history/output pane**.
- For failures: check the **activity-level error message** first, not the whole pipeline log.

2. Semantic Models

Import vs. DirectQuery vs. Direct Lake

This is the single most-tested comparison. Memorize this table:

Mode	Data location	Performance	Freshness
Import	Copied into memory	Fastest	Only as fresh as last refresh
DirectQuery	Stays in source, queried live	Slower (network round-trip)	Always live
Direct Lake	Reads Delta files directly from OneLake (no copy, no query engine round-trip)	Near-Import speed	Live, Fabric-only

Trick: “Millions/billions of rows + Power BI + don’t want to import” = **Direct Lake** (this is basically a guaranteed exam scenario).

Direct Lake Fallback

- If a query is too complex for Direct Lake (e.g., certain DAX functions, or the table exceeds row-group/column limits), it **falls back to DirectQuery** automatically.
- **Trigger phrase:** “cannot be satisfied directly from the lake” → fallback to DirectQuery.

Composite Models

- Mixing storage modes (e.g., one table Import, another DirectQuery/Direct Lake) in the same model.
- **Trigger:** “combine a large fact table with a small reference table refreshed differently.”

Star Schema

- Always the “correct” modeling answer unless stated otherwise. Fact tables = measures/events; Dimension tables = descriptive attributes.
- **Why tested:** improves performance, simplifies DAX, avoids ambiguous relationships (avoid snowflake unless explicitly justified).

Model Optimization Checklist (memorize this list — it’s a recurring theme across multiple questions)

1. Remove unused columns/rows
2. Use correct, smallest data types (e.g., use whole number instead of text for IDs)
3. Disable auto date/time
4. Set appropriate summarization
5. Use star schema
6. Use incremental refresh for large tables
7. Use variables and avoid iterators in DAX where possible

Incremental Refresh

- **Trigger:** “reduce refresh time,” “only refresh recent data,” “large fact table with historical data that doesn’t change.”
- Key concept: define **RangeStart/RangeEnd** parameters; historical partitions are locked, only recent ones refresh.

Partitioning

- Improves refresh AND query performance by letting the engine skip irrelevant partitions/segments.

DAX vs. SQL vs. KQL — pick by scenario:

- **DAX** → semantic model calculations, measures, Power BI visuals.
 - **SQL** → Warehouse/Lakehouse SQL endpoint, relational transformations.
 - **KQL** → Eventhouse / real-time/streaming/log analytics data.
-

3. Security & Maintenance

RLS vs. OLS

- **RLS (Row-Level Security):** restricts *rows* (e.g., a manager sees only their region’s rows).
- **OLS (Object-Level Security):** restricts *entire columns or tables* (e.g., hide the Salary column).
- **Trigger:** “shouldn’t even see the column exists” → OLS. “See only their own data” → RLS.

Implementing RLS

- Create **roles** with DAX filter expressions, then assign users/groups to roles.
- **Trick:** Filter on the **dimension table**, not the fact table — it propagates through relationships and is far more efficient (one filter row vs. millions of fact rows).

Sensitivity Labels

- Inherited from Microsoft Purview; classify data (e.g., Confidential, Public) and can persist when exported to Excel/PDF.

Endorsements

- **Promoted** vs. **Certified** — signals trust/quality of a dataset/report so others discover and reuse it confidently. Certified requires admin/reviewer approval.

OneLake Data Access Security

- Security defined **once** in OneLake/Lakehouse and enforced consistently across every engine that reads it (Power BI, SQL endpoint, Spark) — “define once, enforced everywhere” is the key phrase to recognize.

Workspace Roles

- **Admin > Member > Contributor > Viewer** — know that only Admin/Member can manage permissions; Contributor can edit content but not manage access; Viewer is read-only.

Deployment Pipelines (Dev → Test → Prod)

- **Trigger:** “promote content between environments,” “separate stages for development, testing, production.”
- Use **deployment rules** (e.g., different data source connections per stage) so Prod doesn’t point at Dev data.

Git Integration

- **Trigger:** “version control,” “collaborate on reports/pipelines with branching,” “track changes over time.”
- Connects a workspace to a Git repo (Azure DevOps/GitHub) for source control and CI/CD.

Capacity Monitoring

- Use the **Microsoft Fabric Capacity Metrics app** to track CU (capacity unit) usage, throttling, and overages.
-

Fast Decision Flowchart for Scenario Questions

When you read a scenario, ask in order: 1. **Is it about moving/transforming data?** → Data prep section (Pipeline/Dataflow/Notebook/Lakehouse/Warehouse). 2. **Is it about a Power BI model’s speed/size/freshness?** → Storage mode (Import/DirectQuery/Direct Lake). 3. **Is it about who can see what?** → Security (RLS/OLS/sensitivity labels/roles). 4. **Is it about promoting solutions across environments?** → Deployment pipelines/Git.

Most Pearson-style questions map cleanly to one of these four buckets — identify the bucket before you even read the answer options.

General Exam-Taking Tricks

- **Eliminate absolute-sounding wrong answers:** if an option says “always” or “never” fixes something, it’s often a distractor.
 - **Watch for “least effort” or “minimize cost/maintenance” phrasing** — these usually point to Fabric-native, low-code features (Dataflow Gen2, Direct Lake, shortcuts) over custom code.
 - **When two answers seem right,** pick the one that avoids **data duplication** (shortcuts, Direct Lake) — Fabric’s whole design philosophy centers on “one copy of data, many engines.”
 - Practice actually building a Direct Lake model, an RLS role, and a deployment pipeline hands-on — several questions are worded around the *steps* of doing this, which sticks better after you’ve clicked through it once.
-

Companion guide to your DP-600 question list. Want the 30 Pearson-style scenario questions next, one at a time, without answers? -e

Part 2: Data Preparation — Full Answers & Diagrams

DP-600: Data Preparation — Full Answers & Diagrams

Complete answers to the 12 core data-prep questions, with diagrams for the ones that are spatial/process-based.

1. When should you use Dataflow Gen2 instead of a Data Factory pipeline?

Dataflow Gen2 is built on Power Query and is best when the work is primarily **transformation-focused** and done by someone comfortable with a low-code, visual, step-by-step editor.

Use Dataflow Gen2 when: - The user is a **business analyst / citizen data engineer**, not a professional data engineer. - You need **Power Query transformations** (merge, unpivot, custom columns, data type conversions) with a visual preview of each step. - The output should land as a **table in a Lakehouse/Warehouse** for reuse across multiple reports. - You want transformations that **support query folding** back to the source for performance.

Use a **Data Factory pipeline** instead when: - You need to **orchestrate** multiple activities (copy data, run a notebook, run a stored procedure, send a Teams notification, run a Dataflow) in a defined sequence. - You need **control flow**: conditionals, loops (ForEach), branching on success/failure. - The primary job is **moving data** at scale between systems, not transforming it.

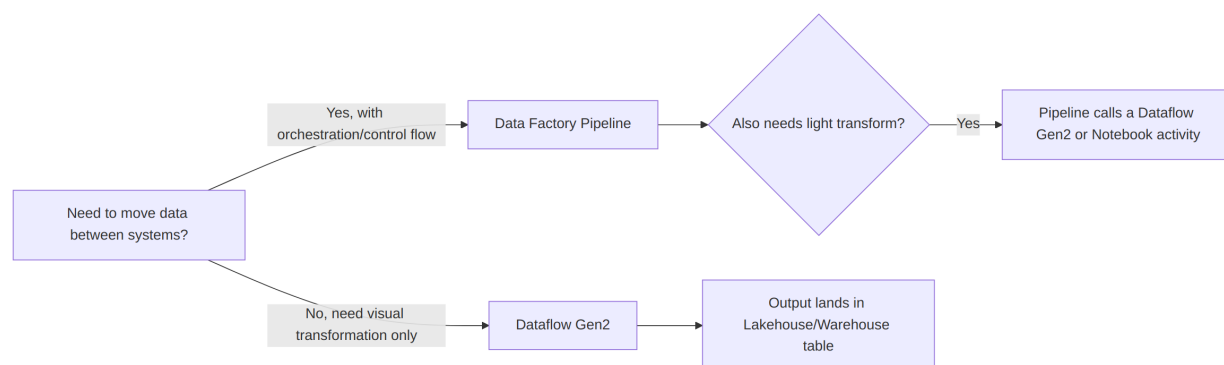


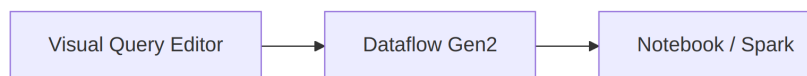
Figure 1: Diagram 0

Exam trigger phrase: "business users need to clean data without writing code" → Dataflow Gen2. "Coordinate multiple steps/systems on a schedule" → Pipeline.

2. When should you use a Notebook for data transformation?

Use a **Notebook (Spark/PySpark, Scala, SQL, or R)** when: - You're working with **large-scale or complex data** that benefits from distributed Spark processing. - The transformation logic is **too complex for visual tools** — custom functions, machine learning, complex windowing, iterative algorithms. - You need **fine-grained control** over performance (partitioning, caching, broadcast joins). - The team consists of **data engineers/data scientists** comfortable with code. - You need to work with **unstructured or semi-structured data** (JSON, images, text files) before it becomes tabular.

Trigger phrase: “data science team,” “PySpark,” “custom Python libraries,” “very large datasets requiring distributed compute” → Notebook.



Rule of thumb hierarchy (simple → complex):

3. What is the difference between a Lakehouse and a Warehouse?

Aspect	Lakehouse	Warehouse
Data structure	Files (Delta Lake format) + tables	Fully relational tables
Engine	Spark (primary) + SQL endpoint (secondary, read-only)	Full T-SQL engine (read/write)
Best for	Data engineers, data scientists, unstructured/semi-structured data, Spark notebooks	SQL developers/analysts, enterprise BI, complex T-SQL (stored procs, multi-statement transactions)
Write access via SQL	✗ No (SQL analytics endpoint is read-only)	✓ Yes (full DDL/DML)
File format exposed	OneLake files + Delta tables	Managed only via SQL
Schema enforcement	Flexible (schema-on-read for files)	Strict relational schema

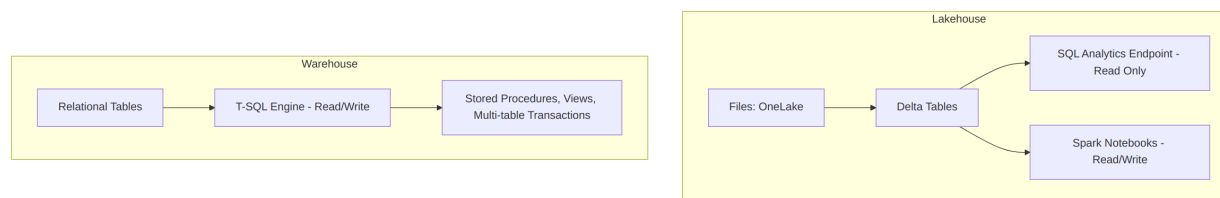


Figure 2: Diagram 2

Trigger phrase: “Needs full T-SQL write capability / stored procedures” → Warehouse. “Needs Spark + flexible file formats” → Lakehouse.

4. What is a OneLake shortcut, and when would you use it?

A **shortcut** is a **virtual reference/pointer** to data stored somewhere else — another Lakehouse/Warehouse in Fabric, or external storage like ADLS Gen2, Amazon S3, or Dataverse — **without physically copying the data**.

When to use it: - Multiple teams/workspaces need the **same data** without duplicating storage or costs. - You want a **single source of truth** so updates to the original data are instantly visible everywhere the shortcut is used. - You need to bring **external data (S3, ADLS)** into your Fabric Lakehouse for unified analytics without an ETL copy step.

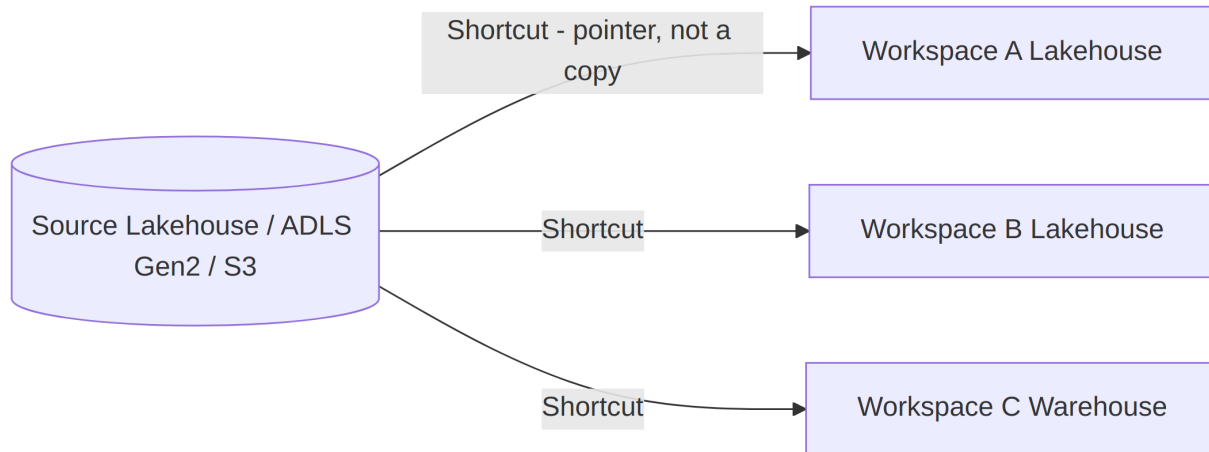


Figure 3: Diagram 3

Trigger phrase: “without copying/duplicating data,” “share data across workspaces/domains,” “bring external cloud storage into Fabric” → shortcut.

5. What is the purpose of the SQL analytics endpoint of a Lakehouse?

The **SQL analytics endpoint** is an **automatically generated, read-only** T-SQL interface over the Delta tables in a Lakehouse. It lets SQL-savvy users and BI tools query Lakehouse data using familiar **T-SQL** — without needing to know Spark or PySpark.

Key points: - Auto-created and auto-updated whenever a table is added/changed in the Lakehouse. - **Read-only** — you cannot INSERT/UPDATE/DELETE through it (that requires a Warehouse). - Enables Power BI, reporting tools, and analysts to connect via a normal SQL connection string.

Trigger phrase: “SQL developers/analysts need to query Lakehouse data without Spark” → SQL analytics endpoint. If the question mentions **writing data back with T-SQL**, that’s a Warehouse, not the endpoint.

6. How do you ingest batch data into Microsoft Fabric?

Batch ingestion options, from simplest to most flexible:

1. **Pipeline Copy activity** — connect to a source (SQL Server, ADLS, SharePoint, on-prem via gateway, etc.) and copy data on a **schedule or trigger** into a Lakehouse/Warehouse.
2. **Dataflow Gen2** — ingest + transform in one step using Power Query connectors.
3. **Notebook** — programmatically read from a source (e.g., `spark.read`) and write to a Lakehouse table.
4. **Shortcuts** — for data you don’t need to physically move at all.

Scheduling options: time-based schedule, tumbling window trigger, or event-based trigger (e.g., file arrival in storage).

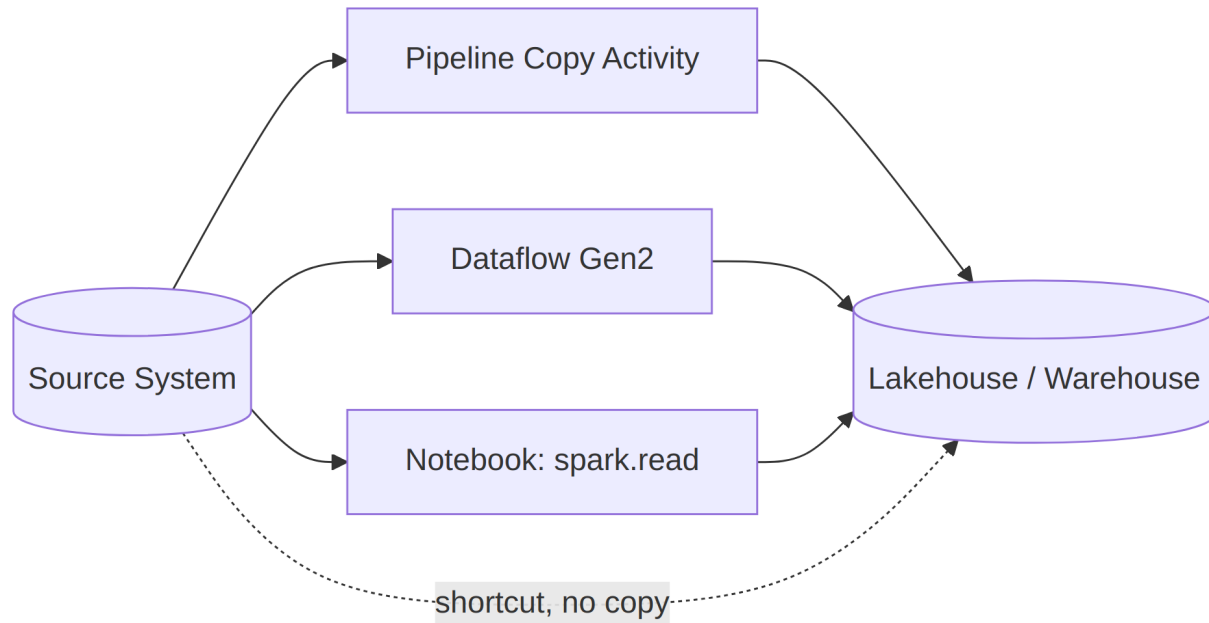


Figure 4: Diagram 4

7. How do you handle null values, duplicates, filtering, joins, and aggregations?

This is usually tested through **which tool** does it, plus the **concept**:

Task	Dataflow Gen2 / Power Query	Notebook (PySpark)	SQL (Warehouse/SQL endpoint)
Remove nulls	"Remove Empty" / Replace Values step	df.dropna() / df.fillna()	WHERE col IS NOT NULL
Remove duplicates	"Remove Duplicates" step	df.dropDuplicates()	SELECT DISTINCT
Filtering	Filter Rows step	df.filter(condition)	WHERE clause
Joins	Merge Queries step	df1.join(df2, on=...)	JOIN clause
Aggregations	Group By step	df.groupBy().agg()	GROUP BY + aggregate functions

Key concept for the exam: know that these are **standard data-cleaning operations** available in all three environments, and picking the "right" one on the exam usually comes back to Question 1/2's logic (who's doing it, what scale, what tool is already in play in the scenario).

8. When should you use Visual Query Editor?

The **Visual Query Editor** (used in the Warehouse/Lakehouse SQL experience or within Dataflows) lets users build **queries via a graphical, no-code interface** (drag tables, pick joins, add filters) rather than writing raw T-SQL.

Use it when: - The user is **not comfortable writing SQL** but needs to explore/query relational data. - You want a **quick, visual way to prototype a query** (joins, filters, aggregations) before exporting it as SQL or a view. - Business users need **self-service data exploration** without engineering support.

Trigger phrase: “user unfamiliar with SQL syntax needs to build a query visually” → Visual Query Editor.

9. What is query folding, and why is it important?

Query folding is the process where Power Query (in a Dataflow Gen2 or Power BI query) **translates your applied steps back into a single query executed at the source** (e.g., pushing a filter/join down into SQL Server) instead of pulling all raw data into Fabric first and processing it there.

Why it matters: - **Performance:** the source system (often optimized for this, like a SQL database) does the heavy lifting instead of Fabric compute. - **Reduced data movement:** only the filtered/aggregated result is transferred, not the entire raw table. - **Cost/efficiency:** less compute consumed in Fabric capacity.

What breaks folding: certain steps (custom M code, adding custom columns with complex logic, merging queries from different sources, renaming late in odd ways) can **prevent folding**, forcing all processing to happen locally in Fabric — which is slower.

Trigger phrase: “improve dataflow refresh performance,” “reduce data transferred from source” → check/preserve query folding.

10. How would you design a pipeline that copies data → transforms it → loads it into a Lakehouse? (Medallion Architecture)

This is the classic **Bronze → Silver → Gold (Medallion)** pattern, orchestrated by a pipeline:

1. **Copy activity:** pulls raw data from the source system into a **Bronze** (raw) Lakehouse table — no transformation, data as-is.
2. **Dataflow Gen2 or Notebook activity:** cleans, validates, deduplicates, joins the raw data → writes into a **Silver** (cleansed/conformed) Lakehouse table.
3. **Another Notebook/Dataflow/Stored Procedure activity:** applies business logic, aggregations, star-schema modeling → writes into a **Gold** (curated, reporting-ready) Lakehouse/Warehouse table.
4. **Semantic model** is built on top of the Gold layer for Power BI reporting.

The whole flow is wrapped in **one pipeline** using sequential activities, each depending on the success of the previous one.

Trigger phrase: “raw → cleaned → curated” or “bronze/silver/gold” → this exact pipeline design.

11. How do you schedule and monitor a pipeline?

Scheduling: - Set a **schedule trigger** (e.g., daily at 2 AM) directly on the pipeline. - Use a **tumbling window trigger** for time-sliced, dependency-aware runs (e.g., process each hour's

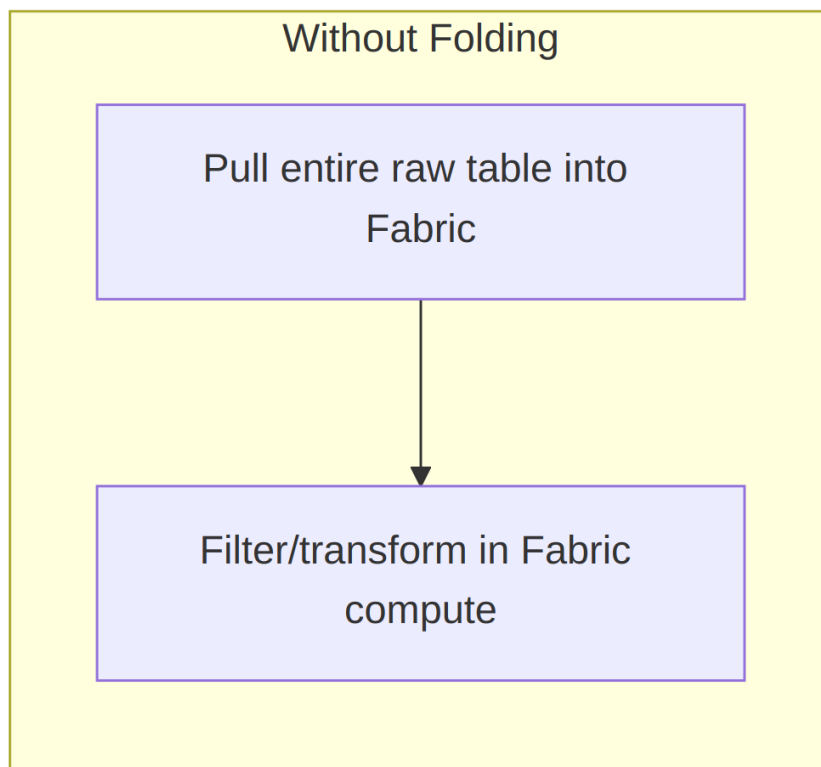
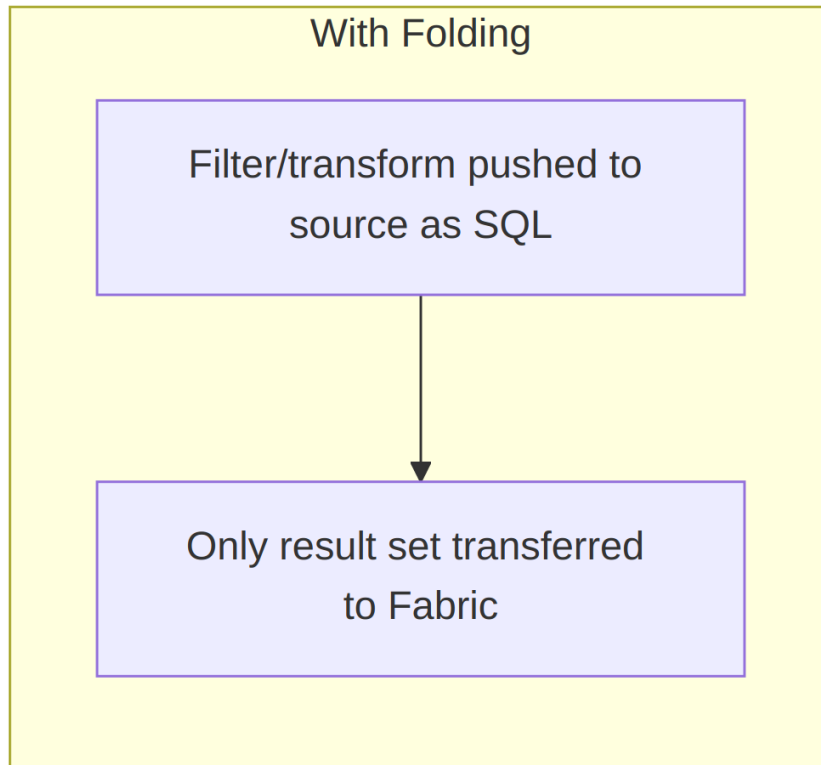


Figure 5: Diagram 5

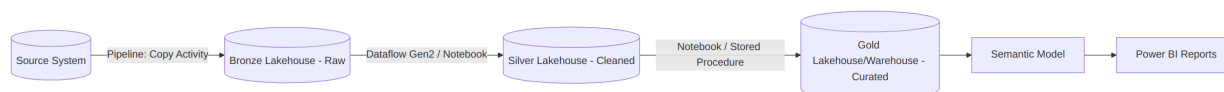


Figure 6: Diagram 6

data exactly once, can retry specific windows). - Use an **event-based trigger** (e.g., run when a new file lands in storage).

Monitoring: - The **Monitoring Hub** in Fabric shows all pipeline runs across workspaces: status (Succeeded/Failed/In Progress), duration, and trigger type. - Drill into a specific run to see **activity-level status** for each step in the pipeline. - Set up **alerts/notifications** (e.g., Teams or Email activity on failure) as part of the pipeline itself.

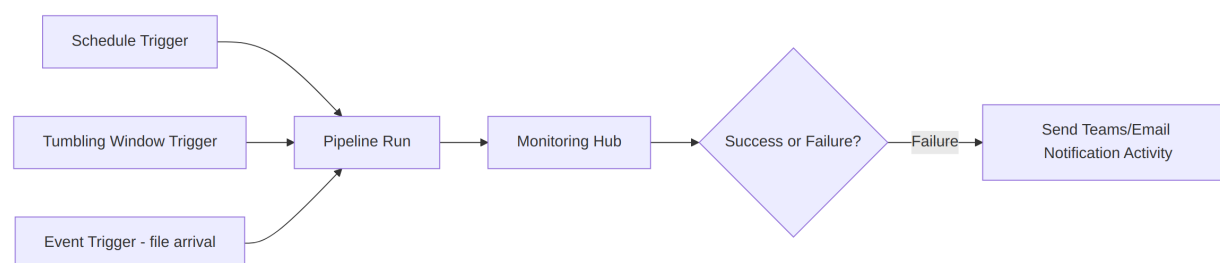


Figure 7: Diagram 7

12. How do you handle pipeline failures and logging?

Failure handling within a pipeline: - Use **activity dependency conditions**: On Success, On Failure, On Completion, On Skip — to branch logic (e.g., “if Copy activity fails, run a notification activity”). - Add a **Try/Catch-style pattern**: main activity → failure path activity (e.g., log error, send alert) connected via the “Failure” dependency. - Use **retry policies** on individual activities (set retry count and interval) for transient failures.

Logging & diagnosis: - Check the **activity run details** in Monitoring Hub — click into the failed run to see the **exact error message** for that specific activity (not the whole pipeline). - For deeper diagnosis of a Copy activity, review the **input/output/duration** panel showing rows read/written and where it stopped. - Optionally log custom details (row counts, timestamps, status) to a dedicated **logging table** in a Lakehouse/Warehouse using a Notebook or Stored Procedure activity at the end of each run, for auditability over time.

Trigger phrase: “pipeline fails intermittently” → retry policy. “Need to be notified when it fails” → failure-path activity with Teams/Email. “Need historical audit trail” → custom logging table.

Quick-Reference Summary Table

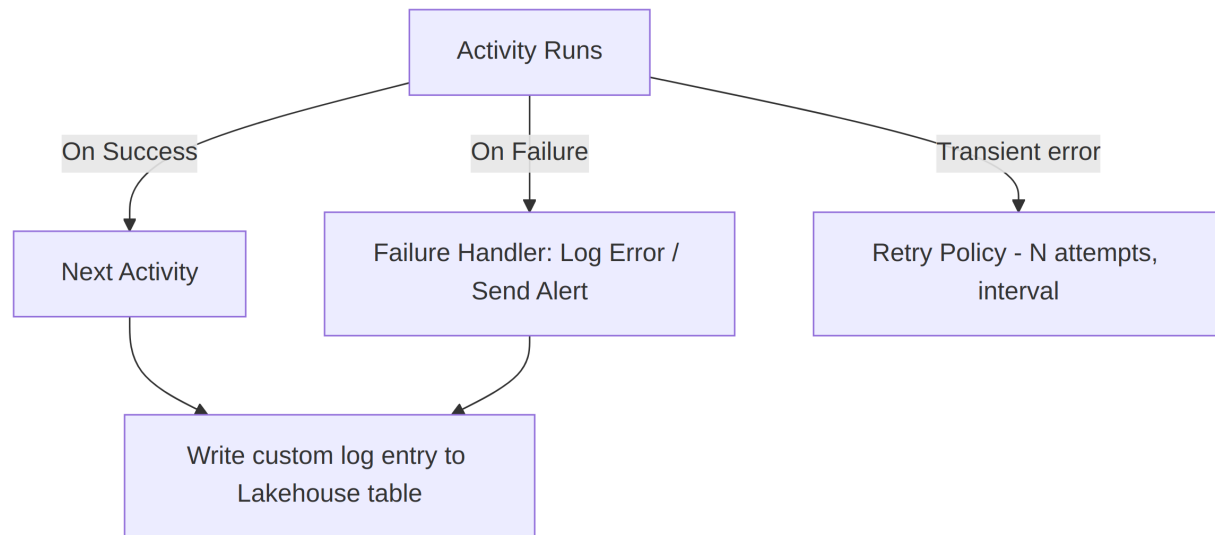


Figure 8: Diagram 8

#	Question	One-line Answer
1	Dataflow Gen2 vs Pipeline	Dataflow Gen2 = visual transform; Pipeline = orchestration/movement
2	When use Notebook	Large-scale, complex, code-based transformations (Spark)
3	Lakehouse vs Warehouse	Lakehouse = files+Spark, read-only SQL; Warehouse = full T-SQL read/write
4	OneLake shortcut	Virtual pointer to data elsewhere, no copy
5	SQL analytics endpoint	Auto-generated, read-only T-SQL view over Lakehouse
6	Batch ingestion	Pipeline Copy, Dataflow Gen2, or Notebook read/write
7	Nulls/dupes/joins/aggs	Same concepts available in Power Query, Spark, and SQL
8	Visual Query Editor	No-code query building for non-SQL users
9	Query folding	Push transformation logic back to source for performance
10	Pipeline design	Bronze → Silver → Gold medallion architecture
11	Schedule & monitor	Triggers (schedule/tumbling/event) + Monitoring Hub
12	Failures & logging	Failure-path activities, retry policies, custom log tables

Part 3: KQL / Microsoft Sentinel — Knowledge Check

Quiz questions covering KQL fundamentals and Microsoft Sentinel visualization, with correct answers marked.

Search & Project Operators

1. What does the search operator do?

- ✓ **Searches across tables and isn't column-specific.**
- Searches only data in the last hour.
- Searches in columns specified.

2. What are project operators?

- Project operators filter a table to the subset of rows that satisfy a predicate.
- Project operators create summarized columns and append them to the result set.
- ✓ **Project operators add, remove, or rename columns in a result set.**

Summarize, Functions & Render Visualizations

3. What is the primary purpose of using the summarize operator in KQL when analyzing log data in Microsoft Sentinel?

- To filter data based on specific criteria
- To visualize data in graphical format
- ✓ **To calculate aggregate values over a dataset**

4. You are tasked with identifying accounts that have logged in more than ten times in the last hour. Which KQL function would be most appropriate for this task?

- ✓ **countif()**
- arg_max()
- dcount()

Why: `countif()` lets you count rows matching a condition (e.g., count logon events per account, then filter where `count > 10`). `arg_max()` returns the row with the maximum value of an expression — useful for “most recent” lookups, not frequency counts. `dcount()` counts *distinct* values, not occurrences per account.

5. Which render visualization type in KQL is best suited for displaying the distribution of security events across different user accounts?

- columnchart
- timechart
- ✓ **piechart**

Why: A pie chart is best for showing proportional distribution across categories (e.g., % of events per account). A time chart is for trends over time, not categorical distribution.

6. Your organization needs to monitor failed logons by computer in a visual format. Which KQL render visualization type should you use to display this data effectively in Microsoft Sentinel?

- areachart
- scatterchart
- ✓ **barchart**

Why: A bar chart is ideal for comparing a metric (failed logons) across discrete categories (computers).

7. When you want to visualize the trend of security events over time in Microsoft Sentinel using the render operator, which visualization type should you choose?

- scatterchart
- ✓ **timechart**
- piechart

Why: “Trend...over time” is the textbook trigger phrase for timechart.

8. Which KQL summarize function would you use to calculate the total sum of data transferred between servers in the past hour?

- avg()
- ✓ **sum()**
- count()

9. You need to create a list of all accounts that have logged into a specific computer using KQL. Which summarize function will achieve this?

- make_set()
- count()
- ✓ **make_list()**

Why: make_list() returns an ordered list of all values (including duplicates) for a column within each summarize group. make_set() returns only *distinct/unique* values. Since the question asks for “a list of all accounts” (not necessarily unique), make_list() is the better fit — watch this distinction closely, as the exam frequently tests make_list() vs. make_set() vs. count()/dcount().

10. Your company wants to visualize the number of logins per day over the past month using KQL in Microsoft Sentinel. Which query will help achieve this goal?

- ✓ **SecurityEvent | where TimeGenerated > ago(30d) | summarize count() by bin(TimeGenerated, 1d) | render timechart**
- SecurityEvent | summarize count() by bin(TimeGenerated, 1d) | where TimeGenerated > ago(30d) | render timechart
- SecurityEvent | where TimeGenerated > ago(30d) | render timechart | summarize count() by bin(TimeGenerated, 1d)

Why — KQL pipe order matters: operators execute top-to-bottom. Correct order is **filter → summarize → render**: 1. where first, to narrow the dataset before heavy processing (more efficient, and TimeGenerated may no longer exist as a raw column after summarizing). 2. summarize ... by bin(...) to bucket counts per day. 3. render timechart last, since it’s a terminal visualization operator — nothing can follow it in the pipe.

11. You need to identify the most recent login event for each user in your organization. Which KQL summarize function will provide this result?

- arg_min()
- max()
- ✓ **arg_max()**

Why: arg_max(TimeGenerated, *) returns the **entire row** corresponding to the maximum value of an expression (e.g., the latest login’s full record) — not just the max value itself, which is all max() would give you. arg_min() is the same concept but for the minimum (earliest) value.

Quick-Reference: KQL Summarize/Render Functions

Function/Operator	Purpose
count()	Number of rows in each group
countif(condition)	Number of rows matching a condition
dcount()	Number of <i>distinct</i> values
sum()	Total of a numeric column
avg()	Average of a numeric column
make_list()	Ordered list of all values (duplicates included)
make_set()	List of <i>unique/distinct</i> values
arg_max(col, *)	Full row where col is at its maximum
arg_min(col, *)	Full row where col is at its minimum
render timechart	Trend over time
render columnchart / barchart	Compare a metric across discrete categories
render piechart	Proportional distribution across categories
render scatterchart	Relationship between two numeric variables

Additional Practice — Set 2

1. To visualize the trend of logon attempts over the past month in Microsoft Sentinel, which KQL render visualization type should you use?

- piechart
- columnchart
- ✓ **timechart**

Why: “Trend...over the past month” = a time-based trend → timechart.

2. When using KQL, which operator would you employ to group login events by computer and count the occurrences?

- render
- ✓ **summarize**
- where

Why: summarize ... by Computer groups rows and lets you apply an aggregate like count(). where only filters rows; render only visualizes an already-computed result.

3. Which KQL render operator visualization type is best suited for showing the number of security incidents by computer?

- piechart
- timechart
- ✓ **barchart**

Why: Comparing a count across discrete categories (computers) → bar-chart/columnchart. Reserve piechart for proportional share of a whole, and timechart for trends over time.

4. Your organization wants to track the number of failed logons for each user in the past 24 hours using KQL in Microsoft Sentinel. Which of the following KQL queries would you use to achieve this goal?

- SecurityEvent | summarize count() by Account | where TimeGenerated > ago(24h) and EventID == 4625
- ✓ **SecurityEvent | where TimeGenerated > ago(24h) | where EventID == 4625 | summarize count() by Account**
- SecurityEvent | summarize count() where EventID == 4625 by Account

Why: Filter first (where), then aggregate (summarize). Filtering after summarize is invalid/inefficient because the raw columns needed for the filter (TimeGenerated, EventID) are no longer available in their original row-level form once summarized, and summarize count() where ... isn't valid KQL syntax — where must precede summarize in the pipe.

5. If you wanted to compare the number of security incidents across different departments in your organization using KQL, which render visualization type would be most effective?

- areachart
- ✓ **barchart**
- timechart

Why: Same pattern as Q3 — comparing a metric across discrete categories (departments) → barchart.

6. You are tasked with identifying accounts that have logged in more than ten times in the last hour. Which KQL function would be most appropriate for this task?

- dcount()
- arg_max()
- ✓ **countif()**

Why: Same as the earlier question in this guide — countif() counts rows matching a condition, or you'd use summarize count() by Account and then filter the aggregated result for > 10.

7. Your organization wants to detect multiple failed login attempts across different applications for a single user. Which KQL query would help achieve this by summarizing application counts per user and IP address?

- ✓ **SigninLogs | where ResultDescription has 'Invalid password' | summarize applicationCount = dcount(AppDisplayName) by UserPrincipalName, IPAddress**
- SigninLogs | summarize count() where ResultDescription has 'Invalid password' by UserPrincipalName, IPAddress
- SigninLogs | summarize count() by UserPrincipalName, IPAddress | where ResultDescription has 'Invalid password'

Why: Filter failed attempts first (where ResultDescription has 'Invalid password'), then use dcount(AppDisplayName) to count the *distinct* applications targeted per user/IP — dcount() is the right choice because the question asks about the number of different applications, not total attempts. The other two options use invalid or inefficient operator ordering (as in Q4).

8. Which KQL summarize function would you use to calculate the total sum of data transferred between servers in the past hour?

- count()
- avg()
- ✓ **sum()**

9. In KQL, what is the effect of using the bin() function with the summarize operator?

- It filters out unwanted rows based on specified criteria.
- It creates a dynamic JSON array of values.
- ✓ **It rounds values down to the nearest multiple, grouping data into specific intervals.**

Why: bin(TimeGenerated, 1d) rounds each timestamp down into 1-day “buckets,” which is exactly how you get one row per day when combined with summarize count() by bin(TimeGenerated, 1d). It's a grouping/bucketing tool, not a filter, and it doesn't produce JSON arrays (that would be make_list()/make_set()/pack_array()).

Microsoft DP-600 · SQL & DAX

Tips & Tricks: T-SQL for Warehouses/Lakehouses and DAX for Semantic Models

A companion to the KQL and Fabric governance notes — the decision rules, syntax traps, and trigger words that come up repeatedly in DP-600's SQL and DAX questions.

Part 1 — T-SQL Tips & Tricks

1. Logical order of operations

SQL is written FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY, but it does not *execute* in the order it's written. Knowing the real execution order explains almost every WHERE-vs-HAVING question on the exam:

Step	Clause	What it operates on
1	FROM / JOIN	Builds the working row set from source tables
2	WHERE	Filters individual rows — before any grouping happens
3	GROUP BY	Collapses rows into groups
4	HAVING	Filters groups — operates on aggregated values
5	SELECT	Chooses/computes output columns (incl. aggregates)
6	ORDER BY	Sorts the final result set

Trap: WHERE SUM(SalesAmount) > 10000 is invalid — WHERE runs before aggregation, so it can't see an aggregate result. Use HAVING instead. This is one of the most repeated mistakes across practice attempts.

2. GROUP BY column-matching rule

Every non-aggregated column in SELECT must appear in GROUP BY, and it should be the *descriptive* column the question is actually asking about — not a surrogate key that happens to be unique per row.

```
-- WRONG: grouping by keys returns one row per key, not per product/year
SELECT p.ProductName, d.Year, SUM(s.SalesAmount)
FROM Sales s
JOIN Product p ON s.ProductKey = p.ProductKey
JOIN Date d ON s.DateKey = d.DateKey
GROUP BY p.ProductKey, d.DateKey -- X mismatched with SELECT list

-- RIGHT
SELECT p.ProductName, d.Year, SUM(s.SalesAmount) AS TotalSales
FROM Sales s
JOIN Product p ON s.ProductKey = p.ProductKey
JOIN Date d ON s.DateKey = d.DateKey
GROUP BY p.ProductName, d.Year
HAVING SUM(s.SalesAmount) > 10000;
```

3. JOIN types — pick by what must survive

Join	Keeps	Trigger phrase
INNER JOIN	Only matching rows on both sides	"matching records in both"
LEFT OUTER JOIN	All left-table rows, matched or not	"every row from Sales even without a match"
RIGHT OUTER JOIN	All right-table rows, matched or not	rare on the exam; equivalent to swapping table order + LEFT
FULL OUTER JOIN	All rows from both sides	"all records from either table"
LEFT ANTI (via merge)	Only left rows with NO match	"exclude records found in another table"

Trap: in the Dataflow Gen2 / Visual Query Editor, "exclude rows found in another table" is a **left anti join**, not an inner join. Inner join would keep only the matches — the opposite of exclusion.

4. Window / ranking functions

Function	Behavior on ties
ROW_NUMBER()	Always unique, sequential — ties get arbitrarily different numbers
RANK()	Ties share a rank; next rank skips (1, 1, 3)
DENSE_RANK()	Ties share a rank; next rank does NOT skip (1, 1, 2)

Trigger phrase: "ties should not create gaps in the next rank" → DENSE_RANK(). "Every row needs a distinct sequence number" → ROW_NUMBER().

```
SELECT ProductName, SalesAmount,
RANK() OVER (ORDER BY SalesAmount DESC) AS Rnk,
DENSE_RANK() OVER (ORDER BY SalesAmount DESC) AS DenseRnk,
ROW_NUMBER() OVER (ORDER BY SalesAmount DESC) AS RowNum
FROM Sales;
```

5. Views vs. stored procedures vs. CTEs

Object	Purpose	Trigger phrase
View	Reusable saved SELECT; no physical copy of data	"rename/reshape columns without changing the underlying table"
Stored procedure	Reusable logic bundling multiple statements/operations	"multi-step database logic," "controlled access pattern"
CTE (WITH ...)	Named temp result scoped to one query, improves readability	"break a complex query into readable steps"
Temp table (#T)	Physically materialized for the session	"reuse an intermediate result across several queries"

6. Security & granular permission syntax

(See the Fabric Security & Governance notes for the full walkthrough — this is the quick syntax reference.)

```
GRANT SELECT ON dbo.SalesReport TO [alice@contoso.com];
DENY SELECT ON dbo.Payroll TO [alice@contoso.com];
REVOKE SELECT ON dbo.SalesReport FROM [alice@contoso.com];

-- Conflict rule: if a user has both a GRANT and a DENY (e.g. via two roles), DENY
always wins.
```

7. Common SQL trigger-word map

Phrase in the question	SQL feature
"total/sum per group, then filter that total"	GROUP BY ... HAVING SUM(...) > x
"filter before aggregating"	WHERE runs first, then GROUP BY
"one row per category with its max/min date"	GROUP BY category with MAX()/MIN(), or window + QUALIFY-style filter
"keep every row from table A regardless of a match"	LEFT OUTER JOIN
"exclude rows that exist in another table"	LEFT ANTI JOIN / NOT EXISTS
"no gaps after a tie in ranking"	DENSE_RANK()
"reusable saved query, no data copy"	CREATE VIEW
"reusable multi-statement logic"	CREATE PROCEDURE

Part 2 — DAX Tips & Tricks

1. Row context vs. filter context

Row context exists while DAX evaluates one row at a time (e.g. inside a calculated column, or inside an iterator like SUMX). **Filter context** is the set of filters already applied by slicers, visuals, or CALCULATE — it's what a measure “sees” when it runs. CALCULATE is the only way to modify filter context from inside a formula.

2. SUMX vs. SUM — the #1 tested DAX mistake

Multiplying two pre-aggregated SUMs is mathematically wrong whenever the multiplied values vary row by row. Use an iterator (SUMX) to compute the expression per row, then sum.

```
-- WRONG: aggregates first, then multiplies the totals
Total Profit = SUM(Sales[Quantity]) * (SUM(Sales[Price]) - SUM(Sales[Cost]))

-- RIGHT: computes profit per line, then sums
Total Profit = SUMX(Sales, Sales[Quantity] * (Sales[Price] - Sales[Cost]))
```

Trigger phrase: “computes correctly per transaction line and aggregates correctly when sliced” → SUMX (or AVERAGEX, MINX, MAXX, COUNTX — same principle, different aggregate).

3. CALCULATE — the core filter-context function

```
Sales YTD = CALCULATE(
    [Total Sales],
    DATESYTD('Date'[Date])
)

Sales 2023 = CALCULATE([Total Sales], Calendar[Year] = 2023)
```

Trap: filter on the small dimension table (Calendar[Year]), not the large fact table (Sales[Year]) — the engine does far less work scanning a small dimension. Prefer a plain boolean filter argument over FILTER(Sales, Sales[Year] = 2023); FILTER() forces a row-by-row scan of the fact table and is slower for a simple equality check.

4. Variables (VAR / RETURN) for performance

A VAR is evaluated once and cached; referencing the underlying measure/expression repeatedly forces DAX to recompute it each time. Declaring a variable but then still calling the original expression forfeits the performance benefit — you must reference the *variable* in every branch.

```
-- WRONG: declares Calc but never uses it
Rating =
VAR Calc = [Variance]
RETURN SWITCH(TRUE(),
    [Variance] > 0.80, "Amazing!",
    [Variance] > 0.60, "Good",
    "Bad")

-- RIGHT: [Variance] is cached once as Calc and reused
Rating =
VAR Calc = [Variance]
RETURN SWITCH(TRUE(),
    Calc > 0.80, "Amazing!",
    Calc > 0.60, "Good",
    "Bad")
```

```
"Bad")
```

5. RANKX and tie behavior

```
Product Rank = RANKX(ALL('Product'[Product Name]), [Sales], , DESC, Skip)
```

Tie-handling argument	Behavior after a tie
Skip (default)	Gapped — like SQL RANK(): two items tied at 1 → next item is rank 3
Dense	No gap — like SQL DENSE_RANK(): two items tied at 1 → next item is rank 2

Trap: a description that only says “tied items share a rank” is incomplete — the exam tests whether you also know what happens to the *next* rank after the tie.

6. USERELATIONSHIP vs. TREATAS

Function	Use when
USERELATIONSHIP()	An inactive physical relationship already exists between two tables (e.g. Date linked to both OrderDate and ShipDate)
TREATAS()	No relationship exists at all between two tables and you need to apply one table's values as a virtual relationship

```
Sales Shipped =  
CALCULATE(  
SUM(Sales[SalesAmount]),  
USERELATIONSHIP('Date'[DateKey], Sales[ShipDateKey])  
)  
  
-- No physical relationship between the two tables:  
Filtered Measure =  
CALCULATE(  
[Total Sales],  
TREATAS(VALUES(Budget[Category]), Sales[Category])  
)
```

7. Storage modes at a glance

Mode	Freshness	Speed	Key limitation
Import	As of last refresh	Fastest	Data duplicated in memory; refresh delay
DirectQuery	Always live	Slower (round-trip per query)	Query performance depends on source
Direct Lake	Live, Fabric-only	Near-Import speed	Falls back to DirectQuery if the query is too complex or if the table is too large

Direct Lake tables cannot currently be mixed with Import/DirectQuery/Dual for the *same table* in one model — “Import and Direct Lake combined” as a single storage choice is a distractor.

8. Model optimization checklist (recurring theme)

- Remove unused columns/rows before publishing
- Use the smallest correct data type (whole number instead of text for IDs/keys)
- Disable auto date/time to avoid hidden, bloated calendar tables
- Build a proper star schema; denormalize snowflaked dimensions into one table

- Use incremental refresh for large fact tables with stable history
- Prefer variables over repeated measure calls; avoid iterators when a simple aggregate works

9. Common DAX trigger-word map

Phrase in the question	DAX feature
"per-row calculation before aggregating"	Iterator: SUMX / AVERAGEX / MAXX / MINX
"share one time-intelligence pattern across many measures"	Calculation group
"avoid recalculating the same expression twice"	VAR ... RETURN
"filter on the smaller table for performance"	CALCULATE(..., DimTable[Column] = value)
"activate an existing inactive relationship"	USERELATIONSHIP()
"no relationship exists between the tables"	TREATAS()
"no gaps in rank after a tie"	RANKX(..., Dense)
"must always reflect the latest lakehouse data"	Direct Lake

Study notes based on Microsoft DP-600 practice question patterns — companion to the KQL Hands-On Lab, the Fabric Security & Governance reference, and the DP-600 Exam Strategy guide.

Securing & Governing a Fabric Data Warehouse

A complete study reference covering the four SQL-level security controls, the classification and governance layer above them, the guided exercises, and a glossary — built for thorough review before an exam or a real deployment.

On this page

1. Dynamic Data Masking
2. Row-Level Security
3. Column-Level Security
4. SQL Granular Permissions
5. How the Four Layers Fit Together
6. Classification & Sensitivity Labels
7. Endorsement & Discoverability
8. Monitoring Governance (OneLake Catalog)
9. Guided Exercises — What They Cover
10. Knowledge-Check Answers
11. Glossary
12. Quick Review Checklist

01 Dynamic Data Masking

Dynamic Data Masking (DDM) hides sensitive column values from unauthorized users at query time — the underlying data never changes, only what's displayed. It's a presentation-layer control, applied per column, and it requires no changes to application code: the same query returns real values to an authorized user and masked values to everyone else.

Function	Effect
<code>default()</code>	Full masking based on data type
<code>email()</code>	Shows a fixed pattern, e.g. <code>aXX@XXXX.com</code>
<code>partial()</code>	Custom prefix/suffix, middle masked
<code>random()</code>	Replaces numeric values with a random value in a range

Applying a mask is a single `ALTER TABLE ... ALTER COLUMN ... ADD MASKED WITH` statement per column:

```
-- Mask the email address
```

```

ALTER TABLE Customers
ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');

-- Show only the last four digits of the phone number
ALTER TABLE Customers
ALTER COLUMN PhoneNumber ADD MASKED WITH (FUNCTION = 'partial(0,"XXX-XXX-",4)');

-- Show only the last four digits of the credit card number
ALTER TABLE Customers
ALTER COLUMN CreditCardNumber ADD MASKED WITH (FUNCTION = 'partial(0,"XXXX-XXXX-XXXX-",4)');

```

The `partial()` signature is `partial(prefix_chars, "padding_string", suffix_chars)` — it exposes a fixed number of real characters at the start and/or end and substitutes everything else with the literal padding string you supply. This is why the phone and credit-card examples above look different even though both use `partial()`: the padding string and character counts are tailored to how each value is normally formatted.

Permissions that matter

- `CONTROL` (Admins, Members, and Contributors) can see unmasked data by default — this is a workspace-role permission, not something you grant per column.
- `GRANT UNMASK ON dbo.Customers TO [user@contoso.com];` — lets one specific non-admin user see real values on that object.
- `GRANT ALTER ANY MASK TO [engineer@contoso.com];` — lets someone add or remove masks without needing full admin rights.
- `REVOKE UNMASK ON dbo.Customers FROM [user@contoso.com];` — removes that unmask access again.

Watch for this on the exam: masking hides *values*, not the column's existence or the row itself. A masked user can still run a query against the column and see that a value exists (or count rows, filter on it, etc.) — they just can't read the real contents. Treat DDM as one layer of protection, not a full access control, and pair it with row- or column-level security when the data is genuinely sensitive.

02 Row-Level Security (RLS)

Filters which rows a user can see in a shared table, based on a predicate function tied to their identity. Everyone queries the same table and the same T-SQL — the engine silently applies the filter underneath, so no application logic needs to change.

Worked example: a shared `Sales` table where each salesperson should see only their own orders.

```

CREATE TABLE [Sales] (
    SalesID INT,
    ProductID INT,
    SalesPerson NVARCHAR(50),
    OrderQty INT,
    UnitPrice MONEY
);

INSERT INTO [Sales] VALUES (1, 3, 'alice@contoso.com', 5, 10.00);
INSERT INTO [Sales] VALUES (2, 4, 'alice@contoso.com', 2, 57.00);

```

```
INSERT INTO [Sales] VALUES (3, 7, 'bob@contoso.com', 4, 23.00);
INSERT INTO [Sales] VALUES (4, 2, 'bob@contoso.com', 2, 91.00);
INSERT INTO [Sales] VALUES (5, 9, 'carol@contoso.com', 5, 80.00);
```

Three pieces go into place, in order:

1. **A dedicated schema** for security objects, so predicate functions and policies are easy to find and lock down separately from regular data objects.
2. **A filter predicate** — an inline table-valued function that returns a row only when its logic evaluates to true for the calling user.
3. **A security policy** that attaches the predicate to a table as a `FILTER PREDICATE` and turns it on.

```
-- 1. Create a schema for security objects
CREATE SCHEMA [Sec];
GO

-- 2. Define the filter predicate
CREATE FUNCTION sec.tvf_SecurityPredicateBySalesPerson(@SalesPerson AS NVARCHAR(50))
    RETURNS TABLE
WITH SCHEMABINDING
AS
    RETURN SELECT 1 AS result
        WHERE @SalesPerson = USER_NAME()
            OR USER_NAME() = 'salesadmin@contoso.com';
GO

-- 3. Apply the policy to the Sales table
CREATE SECURITY POLICY sec.SalesPolicy
ADD FILTER PREDICATE sec.tvf_SecurityPredicateBySalesPerson(SalesPerson) ON [dbo].[Sales]
WITH (STATE = ON);
GO
```

With this policy active, `alice@contoso.com` sees only her two rows, `bob@contoso.com` sees only his, and `salesadmin@contoso.com` sees all five — because the predicate's `OR` clause explicitly carves out an exception for that account. To pause the policy without dropping it (useful while troubleshooting), set `STATE = OFF`.

Two things RLS does not do automatically:

- **It filters everyone, including admins.** RLS predicates apply to workspace Admins, Members, and Contributors the same as any other user. If the predicate has no explicit admin condition, admins get their rows filtered too — write the bypass in yourself if full visibility is required.
- **It's vulnerable to side-channel attacks.** A carefully crafted query can reveal a filtered value without ever directly reading it — for example, forcing a divide-by-zero error only when a hidden value meets a certain condition, then using the presence or absence of the error as a signal. Combining RLS with column-level security and dynamic data masking reduces this exposure, and access to `ALTER ANY SECURITY POLICY` should be tightly restricted and monitored.

03 Column-Level Security (CLS)

Blocks access to entire *columns* without restructuring the table. A user without permission on a protected column gets a permission error only when they try to

select that specific column — every other column in the same table keeps working normally for them.

Scenario

A healthcare `Patients` table has `PatientID`, `Name`, `Address`, `DateOfBirth`, and `MedicalHistory`. Only clinical staff should be able to read `MedicalHistory`; everyone with a legitimate reason to look up a patient should still see the rest.

Four steps set this up:

1. **Identify sensitive columns** — decide which columns need restricted access (here, `MedicalHistory`).
2. **Define roles** — create database roles that map to job functions, e.g. `Doctor`, `Nurse`, `Receptionist`, `Patient`.
3. **Assign users to roles** — each person is placed in the role matching their job.
4. **Apply GRANT then DENY** — grant broad table access to everyone who needs the table at all, then explicitly deny the sensitive column to the roles that shouldn't see it.

```
-- Create roles
CREATE ROLE Doctor AUTHORIZATION dbo;
CREATE ROLE Nurse AUTHORIZATION dbo;
CREATE ROLE Receptionist AUTHORIZATION dbo;
CREATE ROLE Patient AUTHORIZATION dbo;
GO

-- Grant SELECT on all columns to all roles
GRANT SELECT ON dbo.Patients TO Doctor;
GRANT SELECT ON dbo.Patients TO Nurse;
GRANT SELECT ON dbo.Patients TO Receptionist;
GRANT SELECT ON dbo.Patients TO Patient;
GO

-- Deny SELECT on MedicalHistory to roles that shouldn't see it
DENY SELECT ON dbo.Patients (MedicalHistory) TO Receptionist;
DENY SELECT ON dbo.Patients (MedicalHistory) TO Patient;
GO
```

Result: `Doctor` and `Nurse` can read every column including `MedicalHistory`. `Receptionist` and `Patient` can read `PatientID`, `Name`, `Address`, and `DateOfBirth` fine, but get a permission error the moment a query touches `MedicalHistory` — including `SELECT *`.

Direct Lake note: when Power BI queries a warehouse in Direct Lake mode and column-level security is applied to a table, queries against that table automatically fall back to Direct Query mode. Security is still fully enforced — it's query performance that changes, not correctness.

04 SQL Granular Permissions

Workspace roles and item permissions handle broad access to a Fabric warehouse; granular permissions give you finer control over exactly which users can read a specific table, insert into a specific object, or run a specific stored procedure. Fabric warehouses support the standard `GRANT`, `DENY`, and `REVOKE` statements.

Conflict rule: if a user ends up with both a `GRANT` and a `DENY` on the same permission — say, from two different roles they belong to — **DENY always wins**.

Table and view permissions

Permission	What it allows
SELECT	Read data from the object
INSERT	Add new rows
UPDATE	Modify existing rows
DELETE	Remove rows

```
GRANT SELECT ON dbo.SalesReport TO [alice@contoso.com];
DENY SELECT ON dbo.Payroll TO [alice@contoso.com];
```

Any of these four can also be scoped to specific columns within a table or view — which is exactly how they pair with column-level security.

Function and stored procedure permissions

Permission	What it allows
EXECUTE	Run the function or stored procedure
ALTER	Modify the object's definition
CONTROL	Full ownership-level rights over the object

```
GRANT EXECUTE ON dbo.usp_GetSalesData TO [bob@contoso.com];
DENY SELECT ON dbo.Sales TO [bob@contoso.com];
```

In many warehouse designs, users touch data only through stored procedures, never directly against tables. Granting `EXECUTE` on the procedures while denying direct table access enforces a controlled access pattern — the procedure's logic becomes the only door in.

Principle of least privilege

- An application that reads through stored procedures needs `EXECUTE` on those procedures — not `SELECT` on the tables behind them.
- A reporting role that only reads data should get `SELECT` on specific views, not broad table access.
- Temporary elevated access should be revoked the moment the task is done.

This limits the blast radius if a credential is compromised or someone accidentally runs a destructive query — they can only do what their narrow grant allows.

05How the Four Layers Fit Together

These controls aren't alternatives — they stack, each covering a gap the others

leave open.

Layer	Restricts	Granularity	Main weakness it leaves
Dynamic Data Masking	Column <i>values</i> shown	Column	Existence/structure still visible; not a real access barrier
Row-Level Security	Which <i>rows</i> are visible	Row	Side-channel leakage; applies to admins unless excluded explicitly
Column-Level Security	Whether a <i>column</i> can be queried at all	Column	All-or-nothing per column; no partial reveal
Granular Permissions	Which <i>objects</i> (tables, views, procs) a user can touch, and how	Object / column / action	Doesn't filter rows or mask values by itself

A realistic layered setup for the `Patients` example: CLS denies `Receptionist` from ever selecting `MedicalHistory`; RLS could further limit a `Nurse` to only patients on their ward; DDM masks a phone number so support staff see the last four digits without needing full column access; and granular permissions ensure the reporting application only has `EXECUTE` on a vetted stored procedure rather than raw `SELECT` on the table.

06 Classification & Sensitivity Labels

Without classification, nothing stops someone from using sensitive data they shouldn't have. A lakehouse holding salary data sitting in a workspace where any viewer can export it to Excel is exactly the gap classification closes — it makes sensitivity visible before someone opens or exports an item.

What sensitivity labels do

- They're metadata tags that classify Fabric items by sensitivity level — common levels are Public, General, Confidential, and Highly Confidential, defined by your organization along with the policies behind them.
- They're visible in the Fabric portal for every item type, so users can make an informed choice before opening or sharing something.
- They can also enforce access restrictions through protection policies that control who's able to open the item at all, not just see a label.

How labels travel

In a typical flow — lakehouse → warehouse or SQL analytics endpoint → semantic model → report — labels move automatically through **inheritance**. Labeling the lakehouse at ingestion time protects everything built on top of it without relabeling each downstream item by hand. Labels also travel with the data when it's exported through supported paths: Excel, PDF, PowerPoint, and Power BI Desktop files.

When to classify

Classify anything that could cause harm if misused: personally identifiable information (names, addresses), financial data (salaries, account numbers), health records, and internal-only business data. Even exploratory or experimental data benefits from a baseline label — an unlabeled item tells consumers nothing about its sensitivity, which itself creates uncertainty.

If labels aren't set up yet

Sensitivity labels require configuration by your organization's security team. Until that's in place, clear item descriptions, workspace naming conventions, and tags can make sensitivity visible informally — but note that these approaches only inform consumers, they don't enforce anything.

07Endorsement & Discoverability

Consumers need to know which data is trusted and ready to use. Without an explicit signal, a certified production item looks identical to someone's unfinished experiment — a quarterly revenue report built on a stale personal copy instead of the certified source can misdirect real decisions. Endorsement and metadata make that difference visible.

Endorsement levels

Level	Meaning	Who can apply it
Promoted	Tested and ready for team-level use	Any user with write permissions
Certified	Meets organizational quality standards	Authorized reviewers only
Master data	Authoritative source for core data	Authorized reviewers only (data items only)

Nearly all Fabric and Power BI items can be Promoted or Certified — the one exception is Power BI dashboards. Master data endorsement is narrower still: it only applies to items that actually contain data, like lakehouses and semantic models. In a well-governed environment, the *absence* of any endorsement is itself a signal — it's understood to mean personal, experimental, or not yet reviewed.

Metadata for discoverability

- **Descriptions** — appear in the catalog and let consumers evaluate an item without opening it; a good one explains what the data contains and how it's used.
- **Tags** — flexible labels admins define and owners apply, for grouping by project, refresh frequency, fiscal year, or anything else useful; consumers can search and filter by them.
- **Domains and subdomains** — group workspaces by business area (Sales, Finance, Operations); admins create domains and assign workspaces to them, and domains set ownership boundaries and control what appears where in the catalog.
- **Lineage** — traces how data flows from source to destination, so consumers can see whether an item is built on a trusted source.

Good metadata reduces the odds someone builds on the wrong source, or duplicates work that already exists.

08 Monitoring Governance (OneLake Catalog)

Classification, endorsement, and metadata create governance signals on individual items — but gaps between those signals are invisible until something goes wrong: a dataset stays certified after its source goes stale, or someone who changed teams keeps write access to a production lakehouse. The OneLake catalog exists to surface the current state of the estate and let you act before problems reach consumers.

Govern

Shows the governance posture of data you own or manage:

- **Insights** that highlight gaps — items without descriptions, missing sensitivity labels, unendorsed production data.
- **Recommended actions** with guidance on closing each gap, including a filtered list of exactly which items to fix.
- **Reports** breaking down label coverage, endorsement status, and data freshness across the estate.

Fabric admins get tenant-wide insights refreshed daily from admin monitoring storage (so there's a short delay after a change). Data owners see insights scoped to what they own, refreshed when they access the section. Both can be filtered by domain.

Prioritize fixes that hit consumer trust hardest: items without descriptions are invisible in catalog search, missing sensitivity labels leave sensitive data unprotected, and unendorsed production data is indistinguishable from an experiment.

Secure

A centralized view of who can access what across the estate:

- **Workspace roles** — which users/groups have access to each workspace, and at what level.
- **OneLake security roles** — finer-grained access to specific tables and folders within an item, beyond what workspace roles alone give you.
- **Role management** — audit user access and create, edit, or delete security roles from one place, without visiting each item individually.

Classification, endorsement, and security are complementary, not redundant: classification says what's sensitive, endorsement says what's trusted, and security determines who can actually open and use it. When all three line up, consumers get a consistent picture of what to use and what's off-limits.

How this carries into AI: Copilot and data agents prioritize endorsed items over

unendorsed ones, sensitivity labels set the same access boundaries for AI that they set for human users, and detailed metadata gives the AI better context for generating accurate responses.

09 Guided Exercises — What They Cover

Exercise: Secure a warehouse in Microsoft Fabric (~45 min)

Requires a Fabric paid or trial capacity. Walks through applying dynamic data masking, row-level security, column-level security, and SQL granular permissions to a real warehouse. Optional steps use a second Microsoft Entra user account to verify the controls actually work from another identity's point of view — if you can't create a second account, screenshots at the end of each section show the expected result instead. A second account can come from your org's identity admin (via Entra or the Microsoft 365 admin center) or, without an org tenant, by signing up for a Microsoft 365 trial and adding a user there.

Exercise: Govern analytics data in Microsoft Fabric (~30 min)

Requires a Fabric-enabled workspace. Applies governance practices to a lakehouse and semantic model: endorsing items, adding descriptions, and using the OneLake catalog's Govern section to review coverage before and after.

10 Knowledge-Check Answers

1. Which capability auto-propagates a sensitivity label to downstream semantic models and reports?

Downstream inheritance

Labels move automatically along the lineage path from source item to dependent items; nothing needs to be manually reapplied.

2. Best endorsement level for a single authoritative source of product reference data?

Master data

Reserved for the authoritative source of core/reference data, and only applies to data-containing items.

3. Which OneLake catalog section shows recommended actions for missing descriptions/labels?

Govern

Govern is the gap-finding and remediation-guidance section; Secure is about access, Explore is general browsing.

4. How does endorsement affect a data agent choosing between a certified model and an unpromoted dataset?

It prioritizes the certified semantic model

Endorsement status is a signal the agent uses to prefer trusted sources — it isn't ignored, and it isn't restricted only to master data.

5. Effect of a detailed, specific item description versus a generic one on AI agent responses?

It improves the agent's answers

Rich descriptions give the agent context about scope and freshness, producing more accurate, specific responses — this isn't limited to human catalog browsing.

11Glossary

Term	Meaning
Dynamic Data Masking (DDM)	Column-level control that obscures values shown to unauthorized users without changing stored data
Row-Level Security (RLS)	Filters which rows a user sees in a shared table via a predicate function
Column-Level Security (CLS)	Blocks SELECT on specific columns for specific roles/users
Security predicate	The function encoding the filter logic used by an RLS security policy
Side-channel attack	Inferring a hidden value indirectly — e.g. via a forced error — without reading it directly
Least privilege	Giving a user or app only the permissions it needs, nothing more
Sensitivity label	Metadata tag classifying an item's sensitivity level (e.g. Confidential)
Label inheritance	Automatic propagation of a sensitivity label to downstream dependent items
Endorsement	A trust signal (Promoted, Certified, Master data) applied to an item
Lineage	The traceable path showing where data came from and what it feeds into
OneLake catalog	Fabric's estate-wide view for discovering, governing, and securing

Domain /
subdomain

Business-area grouping of workspaces (e.g. Sales, Finance)

12 Quick Review Checklist

- ☐ Can explain the difference between masking (hides values), RLS (hides rows), and CLS (blocks whole columns)
 - ☐ Know the RLS build order: schema → predicate function → security policy
 - ☐ Remember RLS applies to admins too, unless the predicate explicitly excludes them
 - ☐ Know that DENY beats GRANT in a conflict
 - ☐ Can name the three endorsement levels and who can apply each
 - ☐ Know Govern = gaps/recommendations, Secure = access/roles
 - ☐ Understand that labels inherit downstream and travel through exports
 - ☐ Can name all three endorsement exceptions: dashboards can't be endorsed at all; master data only applies to data-containing items
 - ☐ Understand why the four security layers stack rather than substitute for each other
-

Condensed from the Microsoft Learn modules "Secure a Microsoft Fabric data warehouse" and "Govern analytics data in Microsoft Fabric."

Contents

KQL Hands-On Lab Practice — SampleIoTData.Trips

1

KQL Hands-On Lab Practice — SampleIoTData.Trips

15 worked KQL exercises against database ("SampleIoTData").Trips, each with the query, an explanation, and a screenshot of the actual run and result/visual from the KQL editor.

Question 1: Filter trips where trip_distance is greater than 5

```
database("SampleIoTData").Trips
| where trip_distance > 5
| take 5
```

What it does: where filters rows to only those matching the condition; take 5 limits the output to 5 sample rows for a quick preview.

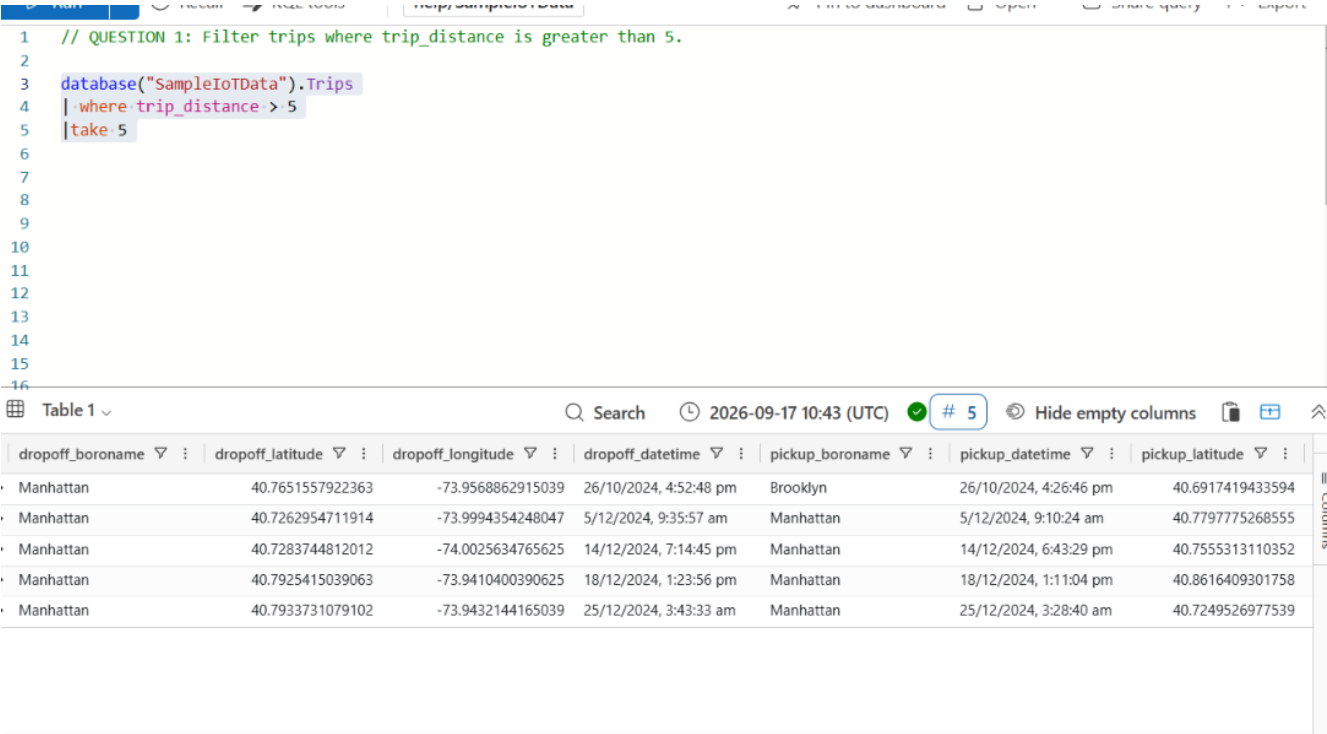


Figure 1: Question 1

Question 2: Display fare_amount and tip_amount for 5 trips

```
database("SampleIoTData").Trips
| project fare_amount, tip_amount
| take 5
```

What it does: project selects only the specified columns; take 5 returns just 5 rows.

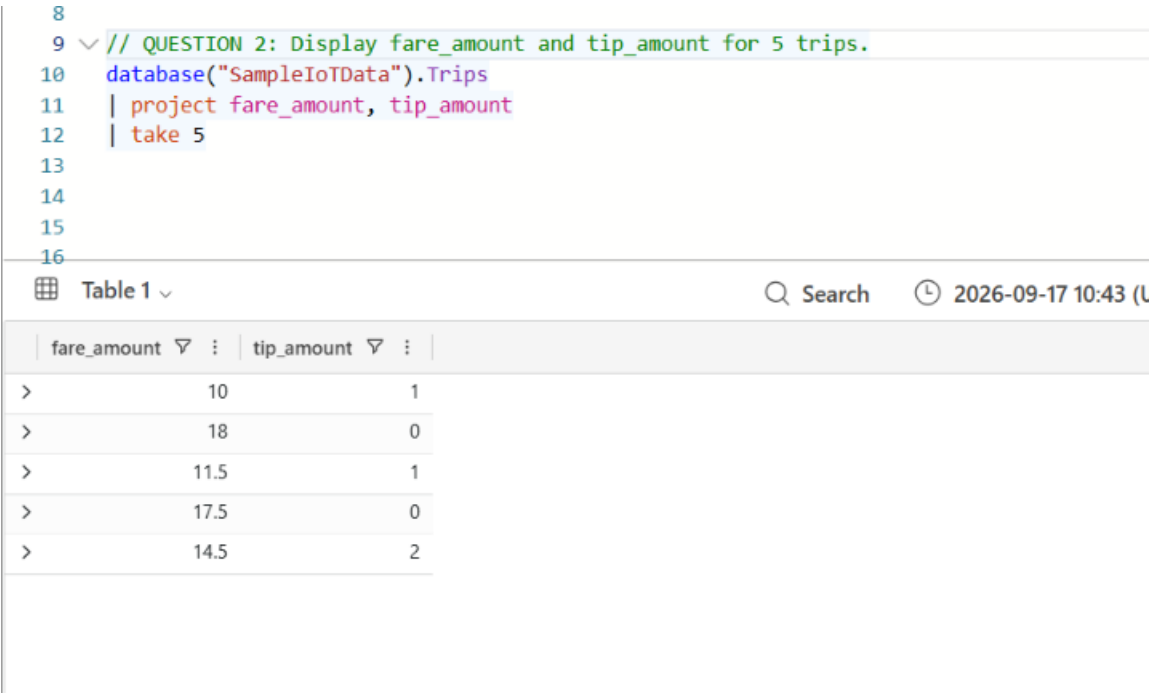
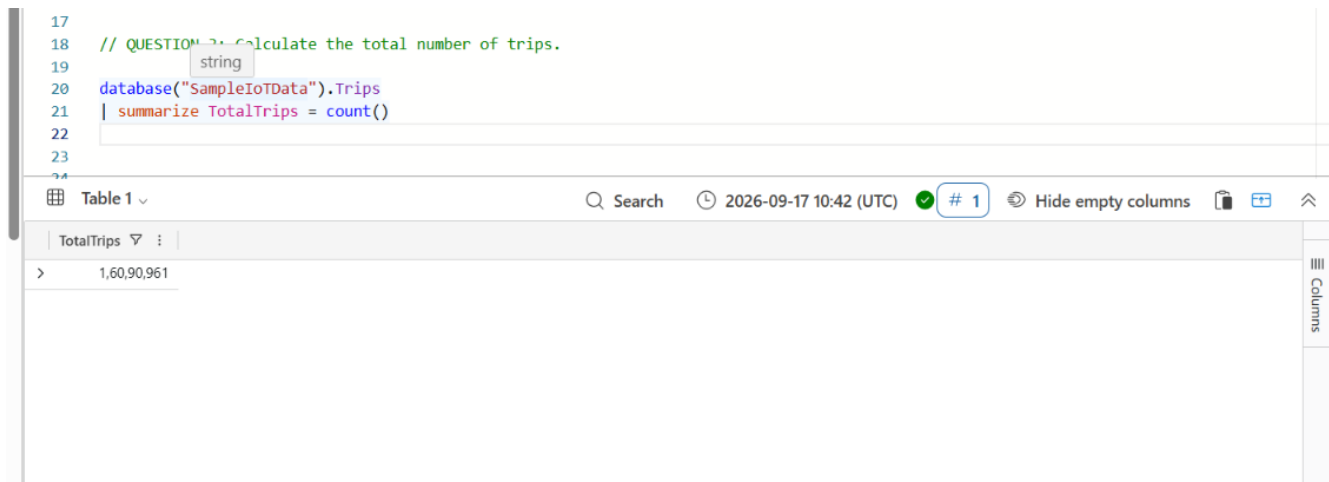


Figure 2: Question 2

Question 3: Calculate the total number of trips

```
database("SampleIoTData").Trips  
| summarize TotalTrips = count()
```

What it does: count() inside summarize (with no by clause) returns a single total row count for the entire table.



The screenshot shows a code editor with a SQL query and a table view below it. The query is: `database("SampleIoTData").Trips | summarize TotalTrips = count()`. The table view shows a single row with the value 1,60,90,961.

TotalTrips
1,60,90,961

Figure 3: Question 3

Question 4: Calculate the average fare for each passenger_count

```
database("SampleIoTData").Trips
| summarize AvgFare = avg(fare_amount)
  by passenger_count
```

What it does: avg() calculates the mean of fare_amount, grouped by passenger_count — one average row per distinct passenger count.

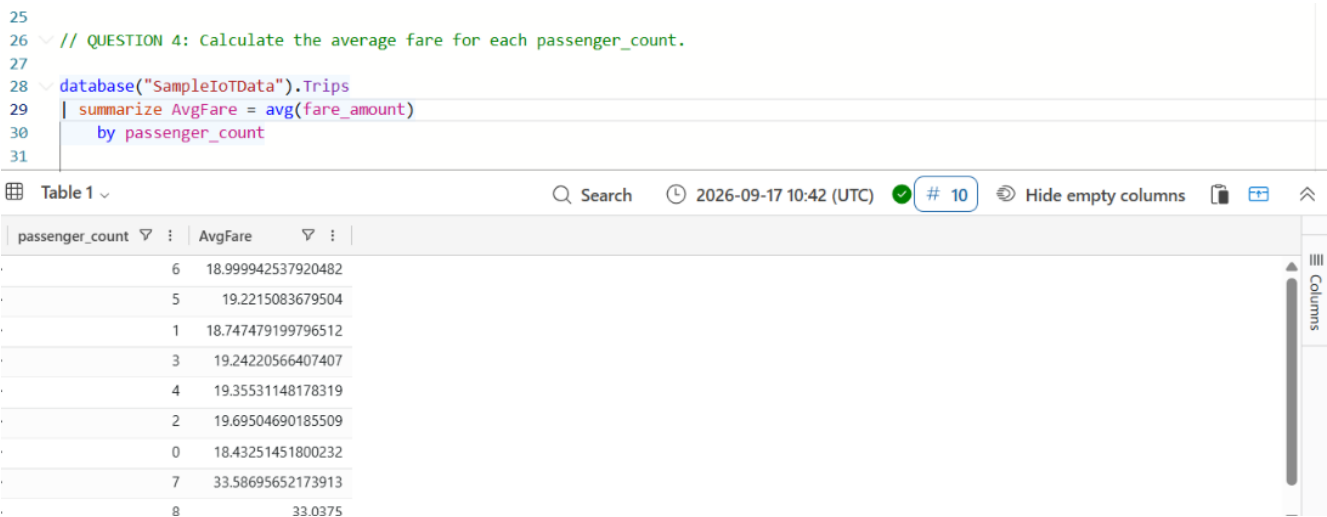


Figure 4: Question 4

Question 5: Count trips for each passenger_count and sort from highest to lowest

```
database("SampleIoTData").Trips
| summarize TotalTrips = count()
  by passenger_count
| sort by TotalTrips desc
```

What it does: groups and counts trips per passenger_count, then sort by ... desc orders the results from highest to lowest.

34
35 // QUESTION 5: Count trips for each passenger_count and sort from highest to lowest.
36
37 database("SampleIoTData").Trips
38 | summarize TotalTrips = count()
39 | by passenger_count
40 | sort by TotalTrips desc
41
42
43
44
45
46
47

Table 1

Search 2026-09-17 10:41 (UTC) # 10 Hide empty columns

passenger_count	TotalTrips
1	1,13,75,610
2	22,65,049
5	8,93,827
3	6,73,720
6	5,56,889
4	3,24,096
0	1,722
7	23
8	17

Figure 5: Question 5

Question 6: Count trips from the last 30 days

```
database("SampleIoTData").Trips  
| where pickup_datetime > ago(30d)  
| summarize TotalTrips = count()
```

What it does: `ago(30d)` computes the timestamp 30 days before “now”; where keeps only trips after that point, then `summarize count()` totals them.

Note: the result was 0 in the screenshot below because the sample dataset’s most recent trips are from mid-2025, which is more than 30 days before the query’s run time (Sept 2026) — the query logic itself is correct.

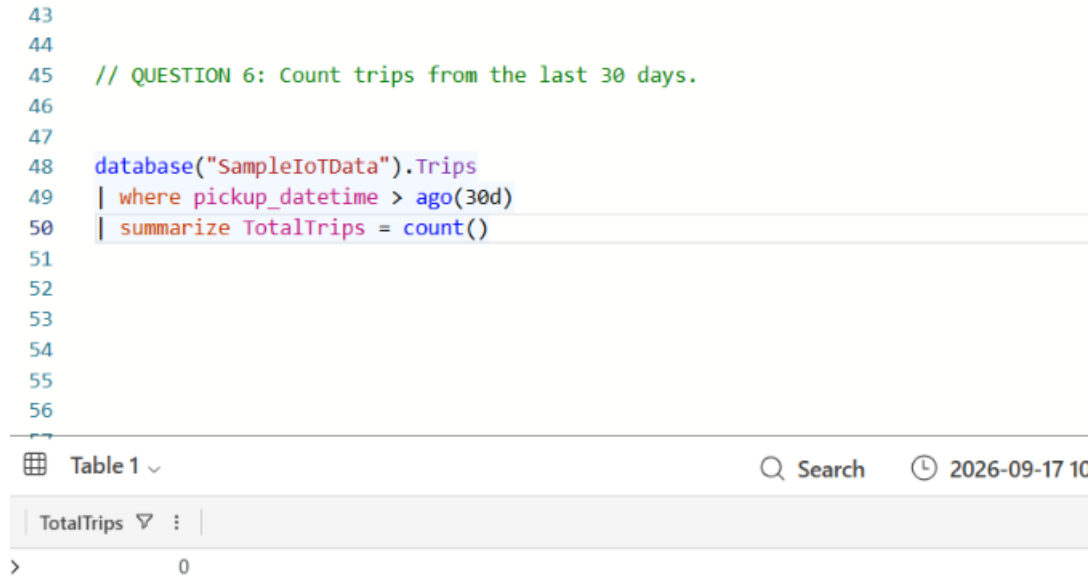
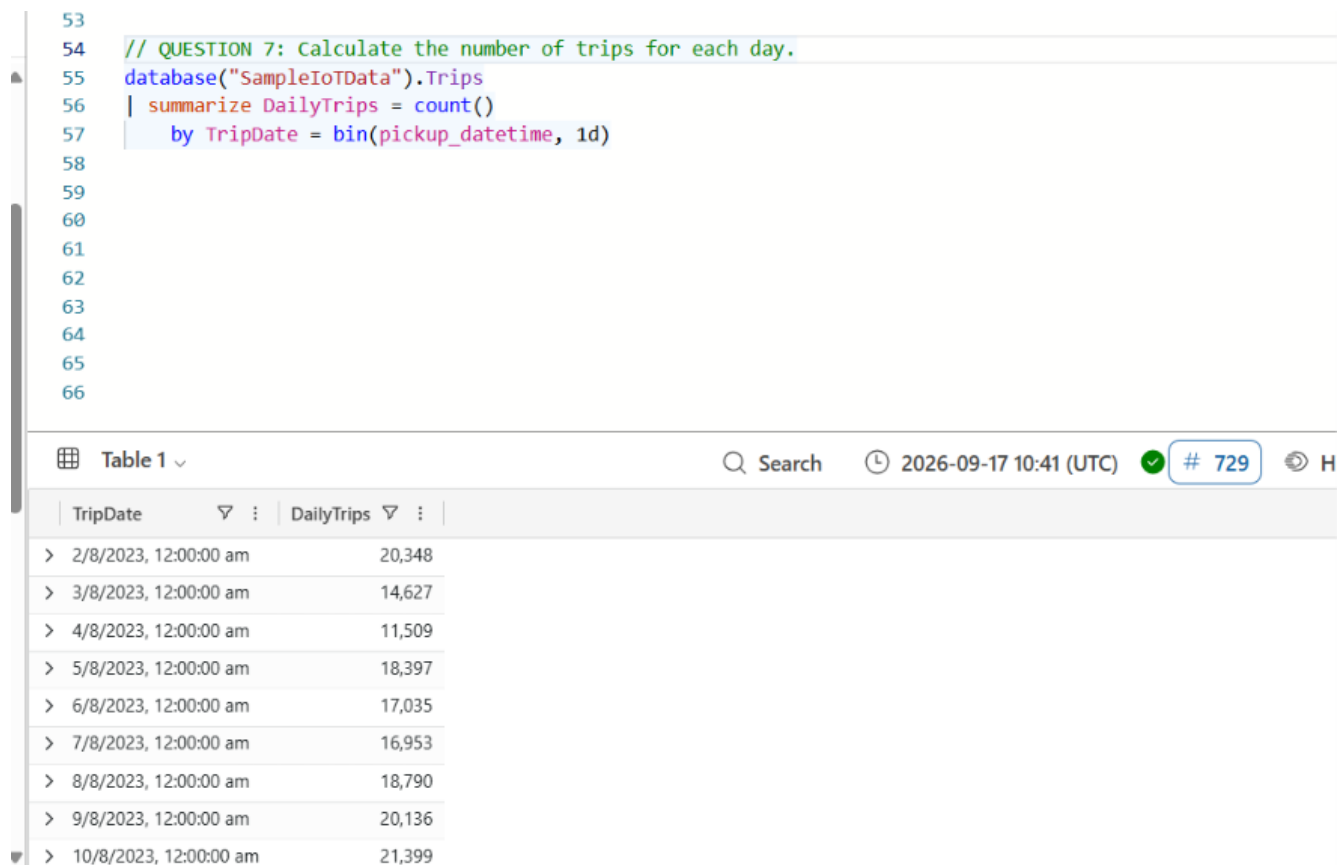


Figure 6: Question 6

Question 7: Calculate the number of trips for each day

```
database("SampleIoTData").Trips
| summarize DailyTrips = count()
  by TripDate = bin(pickup_datetime, 1d)
```

What it does: bin(pickup_datetime, 1d) rounds each timestamp down into 1-day buckets; grouping by that bucket gives one row per calendar day with its trip count.



```
53
54 // QUESTION 7: Calculate the number of trips for each day.
55 database("SampleIoTData").Trips
56 | summarize DailyTrips = count()
57   by TripDate = bin(pickup_datetime, 1d)
58
59
60
61
62
63
64
65
66
```

TripDate	DailyTrips
> 2/8/2023, 12:00:00 am	20,348
> 3/8/2023, 12:00:00 am	14,627
> 4/8/2023, 12:00:00 am	11,509
> 5/8/2023, 12:00:00 am	18,397
> 6/8/2023, 12:00:00 am	17,035
> 7/8/2023, 12:00:00 am	16,953
> 8/8/2023, 12:00:00 am	18,790
> 9/8/2023, 12:00:00 am	20,136
> 10/8/2023, 12:00:00 am	21,399

Figure 7: Question 7

Question 8: Find the most recent trip date

```
database("SampleIoTData").Trips
| summarize DailyTrips = count()
  by TripDate = bin(pickup_datetime, 1d)
| top 1 by TripDate desc
```

What it does: builds the same daily bucket as Question 7, then top 1 by TripDate desc returns only the single most recent day.

```
--
62
63 // QUESTION 8: Find the most recent trip date.
64 database("SampleIoTData").Trips
65 | summarize DailyTrips = count()
66   by TripDate = bin(pickup_datetime, 1d)
67 | top 1 by TripDate desc
68
69
70
71
72
```

Table 1		Search	2026-09-17 10:40 (UTC)
TripDate	DailyTrips		
> 30/7/2025, 12:00:00 am	17,069		

Figure 8: Question 8

Question 9: Count trips where fare_amount is greater than 50

```
database("SampleIoTData").Trips  
| summarize ExpensiveTrips = countif(fare_amount > 50)
```

What it does: countif() counts only the rows where the condition (fare_amount > 50) is true, all in a single aggregate — no separate where step needed.

The screenshot shows a code editor with a SQL query and a table view below it. The query is: `database("SampleIoTData").Trips | summarize ExpensiveTrips = countif(fare_amount > 50)`. The table view shows a single column named 'ExpensiveTrips' with a value of 7,43,568.

ExpensiveTrips
7,43,568

Figure 9: Question 9

Question 10: Find the distinct number of passenger_count values

```
database("SampleIoTData").Trips  
| summarize DistinctPassengerCounts = dcount(passenger_count)
```

What it does: `dcount()` returns an approximate count of *distinct* values in the column — here, how many unique passenger-count values appear in the data.

```
78  
79  
80  
81 // QUESTION 10: Find the distinct number of passenger_count values.  
82  
83 database("SampleIoTData").Trips  
84 | summarize DistinctPassengerCounts = dcount(passenger_count)  
85  
86  
87  
88 // QUESTION 11: Find the complete row with the most recent pickup_datetime.  
89  
90 database("SampleIoTData").Trips  
91 | order by pickup_datetime desc  
92 | limit 1
```

Table 1

DistinctPassengerCounts
10

Figure 10: Question 10

Question 11: Find the complete row with the most recent pickup_datetime

```
database("SampleIoTData").Trips  
| summarize arg_max(pickup_datetime, *)
```

What it does: `arg_max(pickup_datetime, *)` returns the **entire row** where `pickup_datetime` is at its maximum — not just the max value itself. The `*` means “bring back all columns of that row.”

```
87  
88  
89 // QUESTION 11: Find the complete row with the most recent pickup_datetime.  
90  
91 database("SampleIoTData").Trips  
92 | summarize arg_max(pickup_datetime, *)  
93  
94  
95  
96  
97
```

Table 1						
Search 2026-09-17 10:39 (UTC) # 1 Hide empty columns						
pickup_datetime	dropoff_borname	dropoff_latitude	dropoff_longitude	dropoff_datetime	pickup_borname	pickup_latitude
30/7/2025, 11:59:54 pm	Manhattan	40.7601928710938	-73.9946823120117	31/7/2025, 12:12:49 am	Manhattan	40.7903480529785

Figure 11: Question 11

Question 12: Calculate total fare for each passenger_count

```
database("SampleIoTData").Trips
| summarize TotalFare = sum(fare_amount)
  by passenger_count
```

What it does: sum() adds up fare_amount within each passenger_count group.

```
96
97 // QUESTION 12: Calculate total fare for each passenger_count.
98 database("SampleIoTData").Trips
99 | summarize TotalFare = sum(fare_amount)
100   by passenger_count
101
102
103
104
```

Table 1		Search	2026-09-17 10:38
passenger_count	TotalFare		
6	1,05,80,859		
5	1,71,80,703.16		
1	21,32,64,011.8599972		
3	1,29,63,858.799999984		
4	62,72,979.030000005		
2	4,46,10,246.28999997		
0	31,740.789999999997		
7	772.5		
8	264.3		

Figure 12: Question 12

Question 13: Display daily trip counts as a timechart

```
database("SampleIoTData").Trips
| summarize DailyTrips = count()
  by TripDate = bin(pickup_datetime, 1d)
| render timechart
```

What it does: builds the daily trip counts (as in Question 7), then `render timechart` visualizes the trend over time as a line chart.

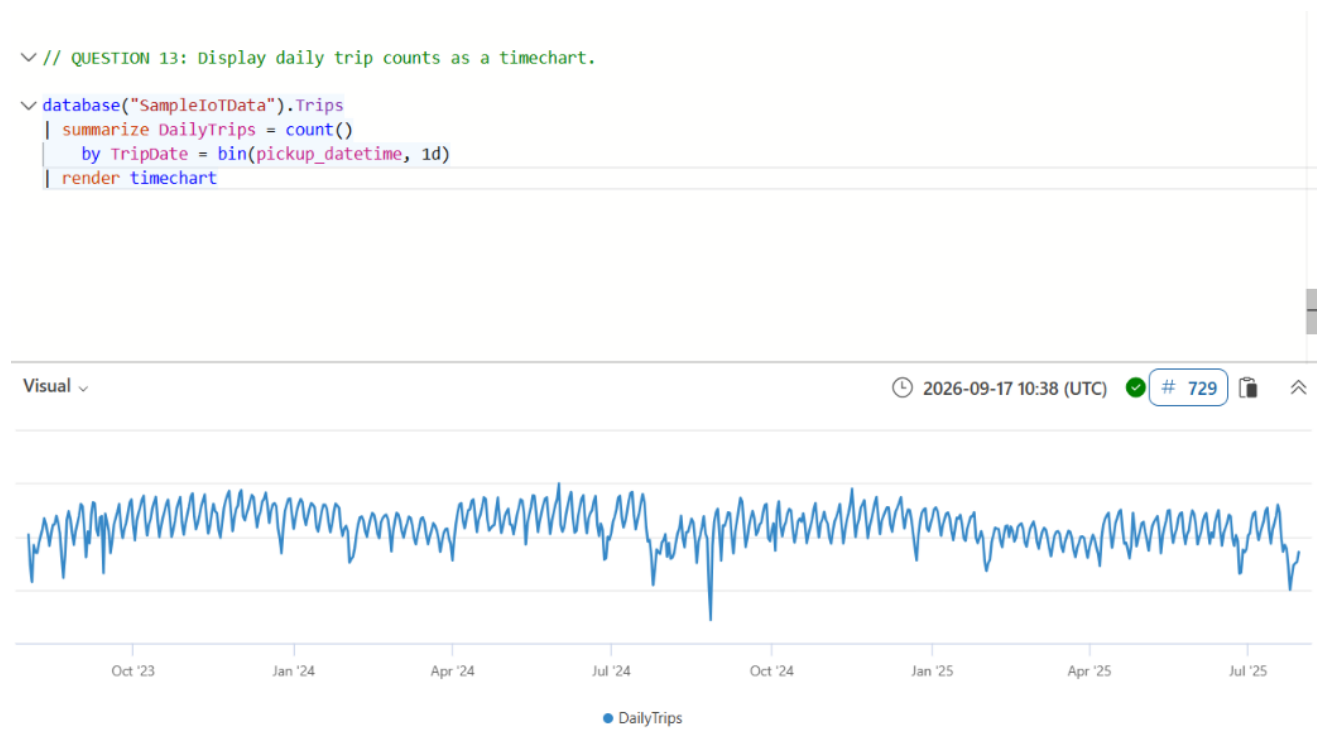


Figure 13: Question 13

Question 14: Display total trips by passenger_count as a bar chart

```
database("SampleIoTData").Trips
| summarize TotalTrips = count()
  by passenger_count
| render barchart
```

What it does: aggregates trip counts per passenger_count, then render barchart compares the totals across each discrete category.

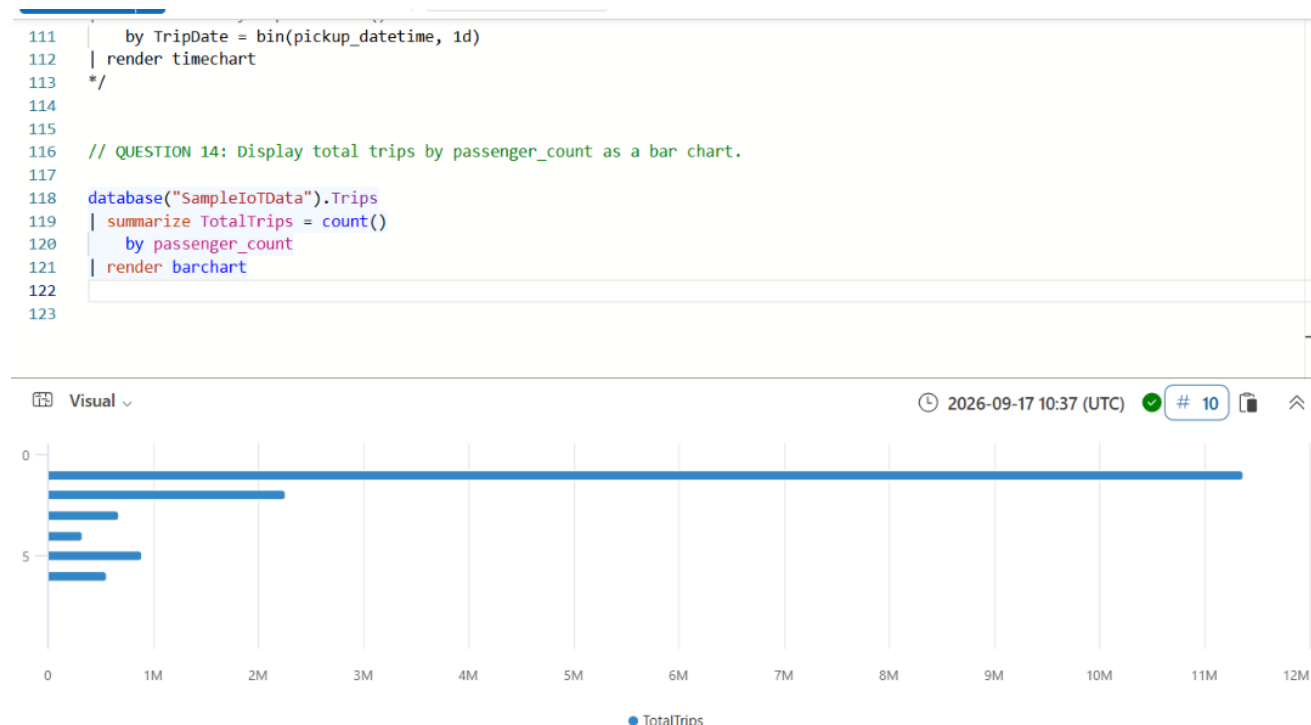


Figure 14: Question 14

Question 15: Display trip distribution by passenger_count as a pie chart

```
database("SampleIoTData").Trips
| summarize TotalTrips = count()
  by passenger_count
| extend PassengerCount = tostring(passenger_count)
| project PassengerCount, TotalTrips
| render piechart
```

What it does: aggregates trip counts per passenger_count, converts the count column to a string with extend ... tostring() (often done so the pie slices label cleanly), keeps just the two needed columns with project, then render piechart shows the proportional distribution across passenger counts.

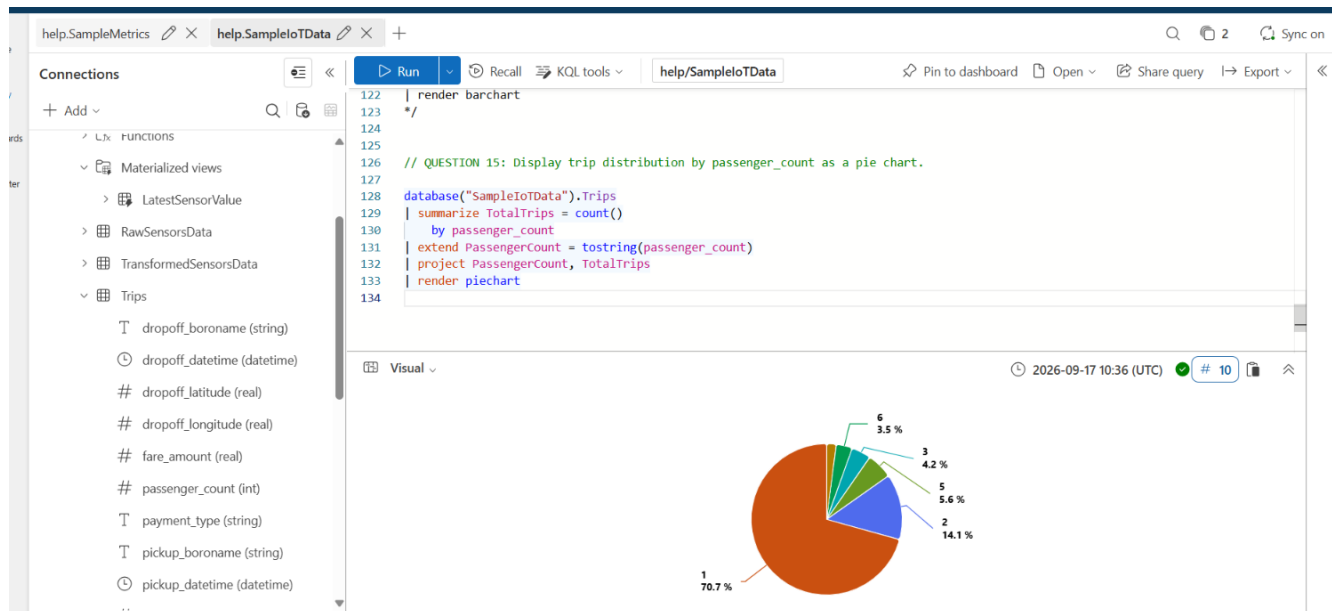


Figure 15: Question 15

Quick-Reference: Operators Used in This Lab

Operator/Function	Role in these exercises
where	Filter rows (Q1, Q6)
project	Select/reshape columns (Q2, Q15)
take	Limit number of rows returned (Q1, Q2)
summarize	Aggregate rows into groups (all of Q3–Q15)
count()	Total row count, overall or per group
avg()	Mean of a numeric column (Q4)
sum()	Total of a numeric column (Q12)
countif()	Conditional count (Q9)
dcount()	Distinct value count (Q10)
arg_max()	Full row at a column's maximum value (Q11)
bin()	Bucket/round values into intervals, e.g. by day (Q7, Q8, Q13)
sort / top	Order results / limit to top N (Q5, Q8)
extend	Add a computed column (Q15)
render timechart	Trend over time (Q13)
render barchart	Compare a metric across categories (Q14)
render piechart	Proportional distribution across categories (Q15)

DP-600 — Prepare the Data

30 High-Level MCQs with Answers

Power BI + Microsoft Fabric | Get Data • Transform Data • Query & Analyze Data

Get Data

1. What should you use to create a connection to an external SQL source?
Answer: B — Data connection
2. Which Fabric capability helps discover organizational data assets?
Answer: A — OneLake catalog
3. Which Fabric capability helps discover real-time and streaming data sources?
Answer: B — Real-Time hub
4. You need to access external data without physically copying it. What should you use?
Answer: A — OneLake shortcut
5. Which Fabric item is designed for files, Delta tables, Spark, and semi-structured data?
Answer: B — Lakehouse
6. Which Fabric item provides a relational environment with full T-SQL read/write capabilities?
Answer: B — Warehouse
7. In an Eventhouse + OneLake architecture, which capability provides integration with OneLake?
Answer: A — OneLake integration
8. You need to move data on a recurring schedule. Which Fabric capability is appropriate?
Answer: A — Pipeline
9. Which capability helps users discover and understand the organization's data?
Answer: A — OneLake catalog
10. You need to access the same data from multiple locations without making another physical copy. What should you use?
Answer: A — OneLake shortcut

Transform Data

11. You need reusable database logic containing multiple SQL statements/operations. What should you use?
Answer: B — Stored procedure
12. You need a reusable query/presentation layer without creating another physical copy of the data. What should you use?
Answer: A — View
13. You create $\text{TotalSales} = \text{Quantity} \times \text{UnitPrice}$. What type of operation is this?
Answer: A — Enriching data
14. A model contains Sales, Product, Customer, and Date tables. Which modeling approach fits this structure?
Answer: A — Star schema
15. You combine descriptive attributes into a fact table to reduce joins. What is this called?
Answer: B — Denormalization
16. You have millions of transactions but reports only need monthly/product totals. What should you use?

Answer: A — Aggregation

17. You need to combine Customer and Order data using CustomerID. Which operation should you use?

Answer: B — Merge/Join

18. Multiple records exist for the same CustomerID when only one should exist. What issue is this?

Answer: B — Duplicate data

19. Some columns contain NULL values that need to be replaced. What operation should you use?

Answer: A — Replace/Fill missing values

20. Numeric values are stored as text. What should you do?

Answer: B — Convert to the appropriate data type

Query & Analyze Data

21. Which tool provides a graphical/no-code way to select, filter, and aggregate data?

Answer: A — Visual Query Editor

22. Which SQL query returns total sales for each customer?

Answer: A — `SELECT CustomerID, SUM(SalesAmount) FROM Sales GROUP BY CustomerID;`

23. Which SQL clause filters rows before aggregation?

Answer: B — WHERE

24. Which SQL clause groups rows for aggregation?

Answer: B — GROUP BY

25. Which KQL query filters trips where trip_distance > 5?

Answer: A — `Trips | where trip_distance > 5`

26. Which KQL operator is used for aggregation?

Answer: C — summarize

27. Which KQL query counts trips by passenger_count?

Answer: B — `Trips | summarize count() by passenger_count`

28. Which KQL function performs a conditional count?

Answer: B — `countif()`

29. Which language is used for dynamic calculations in a semantic model?

Answer: C — DAX

30. Which DAX expression calculates total sales?

Answer: A — `Total Sales = SUM(Sales[SalesAmount])`

Practice result: 28/30 (93.3%) based on your submitted answers.

Revision point: Q7 OneLake integration; Q21 Visual Query Editor.

DP-600 - Data Interpretation Practice

5 scenario questions where you're given an actual result set, chart, or metrics readout and must interpret it — mirrors the exam's habit of showing a query output or visual and asking what it implies, rather than asking you to write the query yourself.

Question 1: Reading a KQL summarize result

You run the following query against SampleIoTData.Trips and get the table below:

passenger_count	AvgFare
1	18.75
2	19.70
6	19.00
7	33.59
8	33.04

Based only on this output, which conclusion is best supported?

- ✓ **A.** Trips with 7 or 8 passengers have a noticeably higher average fare than trips with 1–2 or 6 passengers.
- B.** passenger_count of 7 or 8 is invalid data and should be filtered out.
- C.** The query used countif() to compute AvgFare.
- D.** Fare amount and passenger count are unrelated in this dataset.

Answer: A

Why: The table shows avg(fare_amount) grouped by passenger_count; the 7- and 8-passenger groups average roughly \$33, clearly higher than the ~\$19 seen for 1, 2, or 6 passengers. Nothing in the output indicates invalid data (B), the aggregate shown is an average not a conditional count (C), and the visible pattern is a relationship, not an absence of one (D).

Question 2: Reading a GROUP BY / HAVING result

A T-SQL query groups Sales by SalesPerson and filters with HAVING SUM(OrderQty) > 100. The result set returned is:

SalesPerson	TotalQty
alice@contoso.com	142
carol@contoso.com	118

bob@contoso.com sold 95 total units across several orders. Why does he not appear in this result?

- A.** Bob's rows were removed by the WHERE clause before grouping.
- ✓ **B.** Bob's aggregated total (95) doesn't satisfy the HAVING condition (> 100), so his group is excluded from the final output.
- C.** HAVING only evaluates the first row of each group, and Bob's first order was small.
- D.** SalesPerson must be a foreign key for HAVING to apply, and Bob's isn't.

Answer: B

Why: HAVING filters *after* aggregation, on the grouped result. Bob's group was formed and summed (95), but 95 is not greater than 100, so HAVING excludes that group from the output. There's no indication a WHERE clause removed his rows beforehand, and HAVING evaluates the aggregated value for the whole group, not a single row.

Question 3: Reading a Direct Lake query diagnostic

The Capacity Metrics app shows the following for a semantic model's queries over the last hour:

Table	Storage mode used	% of queries
FactSales	DirectQuery (fallback)	100%
DimDate	Direct Lake	100%

The model was built entirely in Direct Lake mode. What does this readout most likely indicate about FactSales?

- ✓ **A.** FactSales has row-level security or column-level security applied, forcing a fallback to DirectQuery.
- B.** FactSales is missing from OneLake and Fabric substituted DirectQuery automatically.
- C.** DimDate is the only table Direct Lake supports; fact tables always fall back.
- D.** The capacity has run out of memory, so all fact tables fall back permanently.

Answer: A

Why: A Direct Lake table silently falls back to DirectQuery when the query can't be satisfied directly from the lake — common triggers are column-level security on that table, or the query exceeding Direct Lake's row-group/column limits. DimDate querying fine in Direct Lake rules out a capacity- or memory-wide failure (D), and Direct Lake applies to fact and dimension tables alike (C).

Question 4: Reading a Power Query column-quality bar

Power Query's Column quality view for the CustomerEmail column shows:

Valid	Error	Empty
87%	3%	10%

You need every downstream report to treat missing emails consistently before the data is loaded. Which step addresses this reading most directly?

- A.** Use Column distribution to see the most frequent email domain.
- ✓ **B.** Use a Replace Values or Fill Down step to handle the 10% Empty values before loading.
- C.** Delete the CustomerEmail column since 13% of it isn't valid.
- D.** Ignore it — Column quality doesn't reflect the actual loaded data.

Answer: B

Why: Column quality's Empty percentage identifies rows needing a null-handling decision; Replace Values or Fill Down are the standard Power Query steps for that. Column distribution (A) shows frequency, not validity, and 87% valid data is generally too valuable to drop the whole column (C).

Question 5: Reading an RLS test-as-role result

A Sales table has an RLS predicate: WHERE @SalesPerson = USER_NAME() OR USER_NAME() = 'salesadmin@contoso.com'. Testing as three identities returns:

Identity	Rows visible
alice@contoso.com	2 (only her own)
salesadmin@contoso.com	5 (all rows)
workspace Admin (not in predicate)	0 rows

Why does the workspace Admin see 0 rows instead of all 5?

- A. Admins are always exempt from RLS, so this result must be a bug.
- ✓ B. RLS applies to Admins, Members, and Contributors like any other user; since the predicate has no explicit condition for this admin's identity, their rows are filtered out too.
- C. The Admin role overrides RLS but only for read-only queries.
- D. RLS predicates cannot reference USER_NAME(), so the result is undefined.

Answer: B

Why: RLS filters everyone, including workspace Admins, Members, and Contributors — unless the predicate explicitly carves out an exception (as it does for salesadmin@contoso.com). Since this Admin's identity isn't named in the OR clause, they see nothing, which is expected behavior, not a bug.

Study notes based on Microsoft DP-600 practice question patterns — companion to the SQL & DAX tips, KQL Hands-On Lab, and Fabric Security & Governance reference in this pack.

DP-600 Practice Assessment

Full Question Review, Answers & Justifications

Score	36% (18 / 50 correct)
Time Taken	58 minutes
Target Benchmark	78% or higher (per source recommendation)
Report Contents	All 50 questions with your answer, the correct answer, justification, and Microsoft Learn reference — followed by a performance summary and study plan.

Question 1 of 50 | Security & Data Governance**CORRECT**

You must protect sensitive data from unauthorized access in the data warehouse and manage access to specific objects. What action should you take?

Your Answer (correct): Use T-SQL to grant granular permissions.

Justification: T-SQL grants permissions on specific objects, giving true object-level access control. Row-level security is not granular enough; dynamic data masking obscures values but isn't access control; item-level permissions don't reach individual objects.

Reference: learn.microsoft.com/training/modules/secure-data-warehouse-in-microsoft-fabric/5-configure-sql-granular-permissions

Question 2 of 50 | Security & Data Governance**INCORRECT**

User1 needs read-only SQL access to a lakehouse only (not the rest of the workspace, and not Spark access to files). What should you do?

Your Answer: Share the lakehouse with User1 directly and select 'Build reports on the default dataset.'

Correct Answer: Share the lakehouse with User1 directly and select 'Read all SQL Endpoint data.'

Justification: Sharing the lakehouse directly with 'Read all SQL Endpoint data' gives SQL-only, read-only access without workspace membership or Spark file access. Workspace membership would expose all workspace items, including every Power BI report.

Reference: learn.microsoft.com/fabric/data-engineering/lakehouse-sharing

Question 3 of 50 | Security & Data Governance**INCORRECT**

Which three actions protect sensitive customer data while letting authorized users work efficiently?

Your Answer: Grant all users Viewer role; Implement object-level security; Revoke all user permissions temporarily.

Correct Answer: Apply sensitivity labels; Implement object-level security; Use Microsoft Entra ID groups for user permissions.

Justification: Sensitivity labels, object-level security, and Entra ID groups protect data while preserving legitimate access. Revoking all access disrupts work, and a blanket Viewer role can under-provision users who need to edit or manage data.

Reference: learn.microsoft.com/training/modules/secure-data-warehouse-in-microsoft-fabric/4-implement-column-level-security

Question 4 of 50 | Security & Data Governance**CORRECT**

Which three actions limit access to specific data elements in a warehouse without affecting the application layer?

Your Answer (correct): Apply dynamic data masking; Implement column-level security; Use row-level security.

Justification: Column-level security restricts specific columns, row-level security restricts specific rows, and dynamic data masking obscures sensitive values in results — all without changing the application layer. Workspace/item-level roles only govern access to the lakehouse itself, not rows/columns within it.

Reference: learn.microsoft.com/training/modules/secure-data-warehouse-in-microsoft-fabric/

Question 5 of 50 | Security & Data Governance**CORRECT**

You need to control access to data within a report based on user roles. What should you configure?

Your Answer (correct): Configure row-level security.

Justification: Row-level security filters which rows a user can see based on role, providing the needed granularity. Workspace roles are too broad, sensitivity labels address compliance not filtering, and item-level access controls the whole report, not the data within it.

Reference: learn.microsoft.com/training/modules/administer-fabric/4-manage-security

Question 6 of 50 | Analytics Development Lifecycle & Deployment**INCORRECT**

Report authors need a standardized way to start new reports that predefines the SQL Server connection, lets them pick tables, and requires their own credentials. What should you create?

Your Answer: a Power BI Desktop (.pbix) file

Correct Answer: a Power BI data source (.pbids) file

Justification: A .pbids file predefines a connection and launches Desktop with it configured, while authors still authenticate individually and choose tables — with no embedded report/model content. A .pbix, .pbip, or .pbid would carry report/model artifacts that shouldn't be distributed.

Reference: learn.microsoft.com/training/modules/create-manage-power-bi-assets/2-create-reusable

Question 7 of 50 | Analytics Development Lifecycle & Deployment

INCORRECT

You connect Workspace1 to Git (Repo1) for version control. You need multiple users to collaborate on the semantic model and report with minimal development effort. What should you do?

Your Answer: Create a Microsoft Power BI Desktop project and add it to Repo1.

Correct Answer: From Workspace1, create a branch and a new workspace.

Justification: Fabric Git integration lets users branch into their own mapped workspace to work independently, then merge back — the built-in collaboration model. A separate Power BI Desktop project isn't required and adds unnecessary effort; Fabric doesn't connect workspaces to local folders.

Reference:

learn.microsoft.com/training/modules/implement-cicd-in-fabric/3-implement-version-control-and-git-integration

Question 8 of 50 | Analytics Development Lifecycle & Deployment

CORRECT

Which three actions help a deployment pipeline maintain item dependencies and support efficient data refresh?

Your Answer (correct): Configure autobinding; Enable incremental refresh; Use deployment rules for connections.

Justification: Autobinding preserves item connections across stages, incremental refresh keeps refresh policies consistent, and deployment rules manage per-stage data source connections. Static schedules and manual workspace creation don't serve either goal.

Reference: learn.microsoft.com/training/modules/implement-cicd-in-fabric/4-implement-deployment-pipelines

Question 9 of 50 | Analytics Development Lifecycle & Deployment

INCORRECT

Authors need a reusable report template with no embedded data, company branding, preset slicer placement, and a standard layout. What should you create?

Your Answer: a Power BI Project (.pbip) committed to Git

Correct Answer: a Power BI template file (.pbit)

Justification: A .pbit packages layout, visuals, and branding without embedded data — exactly matching a shareable, empty starting template. A .pbip is aimed at source-controlled development and carries full report/model definitions, which isn't the goal here.

Reference: learn.microsoft.com/power-bi/guidance/powerbi-implementation-planning-content-lifecycle-management-develop-manage

Question 10 of 50 | Analytics Development Lifecycle & Deployment

CORRECT

Your organization uses Azure DevOps for code review and needs version control and full change history for workspace items across Dev/PreProd/Prod. Which workspace setting should you enable?

Your Answer (correct): Git integration

Justification: Git integration is the setting that connects a workspace to a repo (e.g., Azure DevOps) for version control, change tracking, and code review. Workspace Identity, Outbound Networking, and Spark settings are unrelated to source control.

Reference: learn.microsoft.com/training/modules/introduction-end-analytics-use-microsoft-fabric/4-use-fabric

Question 11 of 50 | Analytics Development Lifecycle & Deployment

INCORRECT

SalesLH has an internal OneLake shortcut to a table in another lakehouse in the same workspace. You deploy SalesLH to Test via a deployment pipeline. What should you do?

Your Answer: Map the lakehouse dependency in the deployment pipeline to a different upstream lakehouse.

Correct Answer: Create the tables, folders, and files referenced by the shortcut in the target workspace before deployment.

Justification: Internal OneLake shortcuts require the referenced object to already exist in the destination workspace, so the target tables/folders/files must be created there first. Remapping the pipeline's lakehouse dependency doesn't create the missing shortcut targets, and Git integration doesn't resolve this either.

Reference: learn.microsoft.com/fabric/data-engineering/lakehouse-git-deployment-pipelines

Question 12 of 50 | Analytics Development Lifecycle & Deployment

INCORRECT

SalesModel (built on a warehouse fed from a lakehouse) is shared and used by multiple reports. You plan to rename a column and need to see downstream reports/workspaces that might be affected. What should you use?

Your Answer: The SQL analytics endpoint

Correct Answer: data lineage view

Justification: The data lineage view visually maps upstream/downstream dependencies across items, letting you see which reports and workspaces consume the semantic model. The SQL analytics endpoint is for querying data, not for tracing dependency impact.

Reference: learn.microsoft.com/power-bi/collaborate-share/service-dataset-impact-analysis

Question 13 of 50 | Analytics Development Lifecycle & Deployment

INCORRECT

You need to manage large semantic models with external tools. What should you do?

Your Answer: Enable read-only in the XMLA Endpoint settings.

Correct Answer: Enable read-write in the XMLA Endpoint settings.

Justification: External tools that write to a model (e.g., Tabular Editor) require read-write XMLA access; read-only only allows consumption. Large storage format and incremental refresh address size/performance, not write capability.

Reference: learn.microsoft.com/training/modules/create-manage-power-bi-assets/4-dataset-use-xmla-endpoint

Question 14 of 50 | Data Ingestion & Transformation

CORRECT

You need code-free data ingestion into a Fabric warehouse with the ability to apply custom transformations before loading. What should you use?

Your Answer (correct): Dataflows Gen2

Justification: Dataflows Gen2 (Power Query) supports no-code ingestion plus custom transformations before load. Power Automate isn't built for this; Data Factory/notebooks typically require code for custom transforms.

Reference: learn.microsoft.com/training/modules/use-data-factory-pipelines-fabric/3-copy-data

Question 15 of 50 | Data Ingestion & Transformation

INCORRECT

You need to ingest a 1 TB external dataset into a lakehouse with the highest throughput, using a low-code/no-code option. What should you recommend?

Your Answer: Use notebooks and PySpark to load the data to a DataFrame and write results to Lakehouse1.

Correct Answer: Use the Copy data activity of a pipeline to copy the data.

Justification: The Copy data activity is both low-code/no-code and optimized for high-throughput bulk transfer of large datasets. Notebooks/PySpark are efficient but require code, and Dataflow Gen2 is not as efficient for straight bulk copies at this scale.

Reference: learn.microsoft.com/fabric/data-engineering/load-data-lakehouse

Question 16 of 50 | Data Ingestion & Transformation

INCORRECT

You need to ingest 500M+ rows from Azure SQL Database into a lakehouse with no transformations, minimizing cost and admin effort. What should you use?

Your Answer: a SQL stored procedure

Correct Answer: a pipeline with the Copy data activity

Justification: Copy data activity is purpose-built for large, transformation-free ingestion with low overhead. Notebooks suit complex transforms, Dataflow Gen2 suits smaller data, and stored procedures aren't a lakehouse feature (they belong to warehouses).

Reference: learn.microsoft.com/fabric/data-engineering/load-data-lakehouse#considerations-when-choosing-approach-to-load-data

Question 17 of 50 | Data Ingestion & Transformation

CORRECT

Raw data is staged then transformed into final lakehouse tables. What is the most efficient way to run this ETL process?

Your Answer (correct): Use Fabric notebooks.

Justification: Notebooks run on Apache Spark, giving the scale and processing power needed for large-dataset ETL inside a lakehouse. Data Factory pipelines and Dataflow Gen2/Power Query are less efficient here, and eventstreams aren't built for complex batch transformation.

Reference: learn.microsoft.com/training/modules/get-started-lakehouses/3-work-lakehouse

Question 18 of 50 | Data Ingestion & Transformation

INCORRECT

You need to ingest already-formatted forecast data from ADLS Gen2 into a lakehouse with no further transformation, minimizing dev effort and cost. What should you recommend?

Your Answer: Use Dataflow Gen2.

Correct Answer: Use the Copy activity in a pipeline.

Justification: When no transformation is needed, Copy data activity moves data directly and efficiently between source and destination. Dataflow Gen2/notebooks are for when transformations are required, and manual download/upload wastes effort and incurs egress cost.

Reference: learn.microsoft.com/training/modules/use-data-factory-pipelines-fabric/3-copy-data

Question 19 of 50 | Data Ingestion & Transformation

CORRECT

1 TB of legacy accounting data (queried rarely, very selectively) sits in ADLS Gen2. You also need to ingest sales data for regular reporting and support year-end ad-hoc queries on the legacy data, minimizing admin effort/cost. What should you recommend?

Your Answer (correct): Ingest the sales data into the Fabric lakehouse and set up a shortcut to the legacy accounting data in the storage account.

Justification: Since the legacy data is rarely and selectively queried, a OneLake shortcut avoids unnecessary data movement and cost while still enabling ad-hoc queries. Adding a lakehouse SQL endpoint to a warehouse for cross-database queries is a valid but more complex alternative.

Reference: learn.microsoft.com/fabric/onelake/onelake-shortcuts

Question 20 of 50 | Semantic Model Design & Optimization

INCORRECT

You want to denormalize a semantic model to reduce complexity and joins. Which tables should you denormalize?

Your Answer: fact tables on the same level of granularity

Correct Answer: Snowflaked dimension tables

Justification: Snowflaked dimensions are normalized into multiple related tables; denormalizing them into a single dimension table restores a proper star schema and reduces joins. Fact tables of the same grain aren't the source of snowflaking, so denormalizing them doesn't address the join complexity described.

Reference: learn.microsoft.com/power-bi/guidance/star-schema#snowflake-dimensions

Question 21 of 50 | Data Ingestion & Transformation

INCORRECT

In Dataflow Gen2 Diagram view, you need to join FactSales and dimGeography keeping all FactSales rows even without a geography match. What should you do after selecting FactSales?

Your Answer: Use 'Append queries as new' with FactSales first and dimGeography second.

Correct Answer: Use 'Merge queries as new' with join kind 'Left outer', FactSales as left table and dimGeography as right table.

Justification: A left outer merge keeps every row from the left table (FactSales) even when there's no match in the right table (dimGeography) — exactly the join semantics needed. Append is a union-like operation, not a join, so it wouldn't align columns from the two tables.

Reference: learn.microsoft.com/fabric/data-factory/create-first-dataflow-gen2

Question 22 of 50 | Data Ingestion & Transformation

INCORRECT

Using Dataflow Gen2, you must load Sales while excluding any rows found in ExceptionRecords, keeping query folding. Which applied steps should you use?

Your Answer: Merge (inner join), then expand columns.

Correct Answer: Merge (left anti join), then expand columns.

Justification: A left anti join returns only rows from Sales that have no match in ExceptionRecords — precisely the exclusion needed — and expanding columns afterward preserves query folding for performance. An inner join would keep only matching (i.e., excluded) rows, the opposite of the requirement.

Reference: learn.microsoft.com/power-query/merge-queries-overview

Question 23 of 50 | Data Ingestion & Transformation

INCORRECT

You need PySpark code in a notebook to load Customers.parquet into a pandas DataFrame before using Data Wrangler. Which code should you run?

Your Answer: `df = pandas.read_parquet("/lakehouse/default/Files/Customers.parquet")`

Correct Answer: `import pandas as pd`
`df = pd.read_parquet("/lakehouse/default/Files/Customers.parquet")`

Justification: pandas must first be imported (commonly as pd) before it can be used, and the File API path format (/lakehouse/default/Files/...) is required for pandas, as opposed to the relative path Spark uses.

Reference: learn.microsoft.com/fabric/data-science/data-wrangler

Question 24 of 50 | Data Ingestion & Transformation

INCORRECT

You need to save a Spark DataFrame df (with Year and Quarter columns) as a Delta table named Sales, partitioned by Year and Quarter. Which command should you run?

Your Answer:
`df.write.mode("overwrite").format("parquet").partitionBy("Year","Quarter").save("Files/Sales")`

Correct Answer:
`df.write.mode("overwrite").format("delta").partitionBy("Year","Quarter").save("Tables/Sales")`

Justification: format('delta') is required to persist the DataFrame as a Delta table, and it must be saved under Tables/ to register as a lakehouse table — saving to Files/ (or as parquet) would not create a recognized table.

Reference: learn.microsoft.com/fabric/data-engineering/tutorial-lakehouse-data-preparation

Question 25 of 50 | Querying & Analyzing Data**CORRECT**

You need to rename columns for a Power BI report without changing the underlying Delta table. Which warehouse object should you create?

Your Answer (correct): view

Justification: A view encapsulates query logic — such as renaming, filtering, or aggregating columns — while leaving the underlying table completely untouched.

Reference: learn.microsoft.com/sql/t-sql/statements/create-view-transact-sql

Question 26 of 50 | Data Ingestion & Transformation**INCORRECT**

You need to profile data in a notebook DataFrame (df) with minimal administrative effort. What should you do?

Your Answer: Create a pandas DataFrame and visualize it in an embedded Power BI report via QuickVisualize.

Correct Answer: Display the DataFrame by running `display(df)`, then view the Inspect flyout tab.

Justification: The Inspect tab in `display(df)` gives per-column data distribution and statistics with no extra setup. QuickVisualize and the Chart option are for visualization, not statistical profiling.

Reference: learn.microsoft.com/fabric/data-engineering/notebook-visualization

Question 27 of 50 | Querying & Analyzing Data**CORRECT**

In Power Query Editor, you need to identify the percentage of valid records in a column before loading to a report. Which option should you use?

Your Answer (correct): Column quality

Justification: Column quality shows the percentage of valid, error, and empty values in a column. Column distribution shows value frequency, and column profile shows broader statistical detail — neither gives a direct 'percent valid' figure.

Reference: learn.microsoft.com/training/modules/clean-data-power-bi/6-profile-data

Question 28 of 50 | Data Ingestion & Transformation

INCORRECT

You need to identify rows of `df_customers` where any column (axis 1) is NULL. Which statement should you run?

Your Answer: `df_customers[df_customers.isnull().query(axis=1)]`

Correct Answer: `df_customers[df_customers.isnull().any(axis=1)]`

Justification: `.any(axis=1)` checks each row for at least one True (NULL) value across columns, correctly filtering to rows with any NULL. `.query()` is not a valid way to apply this boolean row-wise check.

Reference: learn.microsoft.com/training/modules/explore-analyze-data-with-python/3-exercise-explore-data

Question 29 of 50 | Querying & Analyzing Data

INCORRECT

Complete a SQL query analyzing SalesAmount by ProductName and Year, returning only products with yearly SalesAmount over \$10,000.

Your Answer: GROUP BY p.ProductKey, d.DateKey
HAVING SUM(s.SalesAmount) > 10000

Correct Answer: GROUP BY p.ProductName, d.Year
HAVING SUM(s.SalesAmount) > 10000

Justification: GROUP BY columns must match the non-aggregated SELECT columns (ProductName, Year), not surrogate keys. HAVING filters the aggregated yearly total, which WHERE could not do since it operates on individual rows before aggregation.

Reference: learn.microsoft.com/sql/t-sql/queries/select-having-transact-sql

Question 30 of 50 | Querying & Analyzing Data

INCORRECT

Stakeholders need a reusable, table-like dataset (TotalSales, OrderCount by Region) usable by all workspace users without writing SQL, built in the visual query editor. What should you do?

Your Answer: Use Data preview mode and pin the results.

Correct Answer: Save the query as a view.

Justification: Saving the query as a view creates a persistent, table-like object that any workspace user can query without writing SQL themselves. Pinning a data preview doesn't create a reusable, queryable object.

Reference: learn.microsoft.com/training/modules/get-started-data-warehouse/4-query-transform-data

Question 31 of 50 | Querying & Analyzing Data**CORRECT**

Using the SQL analytics endpoint, you need one row per SalesOrderID with SUM(LineTotal) as a subtotal, returning only groups where the subtotal exceeds 100,000. Which clause filters the aggregated results?

Your Answer (correct): HAVING

Justification: HAVING filters on the result of an aggregation (the summed subtotal), whereas WHERE would only filter individual rows before aggregation.

Reference: learn.microsoft.com/sql/t-sql/queries/select-having-transact-sql

Question 32 of 50 | Querying & Analyzing Data**INCORRECT**

You need ranking of SalesAmount where ties don't create gaps in the next rank (e.g., two rows tied at rank 1, next row is rank 2, not 3). Which window function should you use?

Your Answer: RANK()

Correct Answer: DENSE_RANK()

Justification: DENSE_RANK() assigns ranks with no gaps after ties, matching the required output. RANK() would skip a value (e.g., jump to 3) after a two-way tie at rank 1.

Reference: learn.microsoft.com/sql/t-sql/functions/ranking-functions-transact-sql

Question 33 of 50 | Querying & Analyzing Data**INCORRECT**

You need a DAX measure 'Total Profit' that computes profit per transaction line and aggregates correctly when sliced by product/date. Which expression should you use?

Your Answer: Total Profit = SUM(Sales[Quantity]) * (SUM(Sales[Price]) - SUM(Sales[Cost]))

Correct Answer: Total Profit = SUMX(Sales, Sales[Quantity] * (Sales[Price] - Sales[Cost]))

Justification: SUMX iterates row-by-row, computing profit per transaction line before summing — required because quantity, price, and cost vary per row. Multiplying pre-aggregated SUMs produces mathematically incorrect totals whenever these values vary across rows.

Reference: learn.microsoft.com/training/modules/design-scalable-semantic-models/5-dax-optimizations

Question 34 of 50 | Querying & Analyzing Data**INCORRECT**

In the visual Query Editor, you need to summarize passenger counts by year by joining Trip and Date. Which transformations should you use?

Your Answer: Choose GroupBy on Trip, then use Append queries to join summarized Trip data and Date.

Correct Answer: Choose Merge queries to join Trip and Date, then use GroupBy.

Justification: Merge performs the actual join between Trip and Date (via expanding the matched columns), after which GroupBy can summarize by year. Append performs a union, not a join, so it cannot combine the two tables' columns.

Reference: learn.microsoft.com/fabric/data-warehouse/tutorial-visual-query

Question 35 of 50 | Querying & Analyzing Data**CORRECT**

You need the latest last_stocked_date per category where stock_quantity is under 50. Which SQL statement should you run?

Your Answer (correct): SELECT category, MAX(last_stocked_date) FROM inventory WHERE stock_quantity < 50 GROUP BY category

Justification: Filtering rows first with WHERE, then grouping by category and taking MAX(last_stocked_date), correctly returns one row per category with its latest qualifying stock date.

Reference: learn.microsoft.com/sql/t-sql/queries/select-group-by-transact-sql

Question 36 of 50 | Querying & Analyzing Data**INCORRECT**

You need to analyze recent storm event data in a KQL database using Kusto Query Language. What should you do?

Your Answer: Use a KQL activity in a data pipeline.

Correct Answer: Use a KQL Queryset.

Justification: A KQL Queryset is the direct, purpose-built tool for writing and running KQL queries against a KQL database. A pipeline KQL activity is meant for orchestration, not interactive querying/reporting.

Reference: learn.microsoft.com/training/modules/get-started-lakehouses/3-work-lakehouse

Question 37 of 50 | Semantic Model Design & Optimization

INCORRECT

On an F64 capacity, a lakehouse has 1B historical records plus ~10,000 near-real-time updates every 15 minutes. You need the best report performance while supporting near-real-time reporting. Which semantic model storage mode should you use?

Your Answer: Import and Direct Lake combined

Correct Answer: Direct Lake

Justification: Direct Lake reads data directly from OneLake, giving near-real-time freshness with performance close to Import — without the memory-loading delay of Import or the slower query pattern of DirectQuery. Direct Lake tables cannot currently be combined with Import/DirectQuery/Dual in the same model.

Reference: learn.microsoft.com/power-bi/enterprise/directlake-overview

Question 38 of 50 | Semantic Model Design & Optimization

INCORRECT

Users need a single column chart to analyze SalesAmount by ProductCategory, Year, or CustomerCity, minimizing development effort. What should you do?

Your Answer: Add three overlaid bar chart visuals and use buttons/bookmarks to show one at a time.

Correct Answer: Set up a Fields parameter with ProductCategory, Year, and CustomerCity, and use it in the visual.

Justification: A Fields parameter lets end users swap the field used on a single visual's axis with minimal setup. Bookmarks/buttons to toggle overlapping visuals is a far more effort-intensive workaround.

Reference: learn.microsoft.com/power-bi/transform-model/desktop-what-if

Question 39 of 50 | Semantic Model Design & Optimization

CORRECT

You need logic-based DAX formatting applied only to the Sales measure (not Units or Customers), minimizing admin effort. How should you apply the logic?

Your Answer (correct): Use dynamic measure formatting.

Justification: Dynamic format strings apply conditional formatting logic directly and simply to one specific measure. Calculation groups apply at the visual/page/report level and aren't easily scoped to a single measure.

Reference: learn.microsoft.com/power-bi/create-reports/desktop-dynamic-format-strings

Question 40 of 50 | Semantic Model Design & Optimization

INCORRECT

You're building a composite model with a 500M-row fact table and user-defined aggregations, where most queries hit aggregated data but some need full detail. Which three storage modes should you use?

Your Answer: Aggregated tables: Import; Detailed fact table: DirectQuery; Detailed fact table: Import (contradictory pairing).

Correct Answer: Aggregated tables: Import; Detailed fact table: DirectQuery; Dimension tables: Dual.

Justification: User-defined aggregations require the detailed fact table in DirectQuery, aggregated tables in Import for speed, and dimension tables in Dual mode so they can participate in relationships with both the Import and DirectQuery tables without duplication limits.

Reference: learn.microsoft.com/power-bi/transform-model/aggregations-advanced#storage-modes

Question 41 of 50 | Semantic Model Design & Optimization

CORRECT

On F64, a semantic model sourced from warehouse tables will approach 50 GB once loaded. Which option enables it to refresh in the Power BI service?

Your Answer (correct): Large semantic model storage format

Justification: This setting raises the model size ceiling beyond the default 10 GB compressed limit on Premium/Fabric capacities of F64 or higher — nothing else in the option set changes that limit.

Reference: learn.microsoft.com/training/modules/design-scalable-semantic-models/

Question 42 of 50 | Semantic Model Design & Optimization

CORRECT

You need the same time-intelligence calculations (MTD, YTD, YoY) applied across 50 different measures, minimizing admin effort. What should you do?

Your Answer (correct): Create a calculation group.

Justification: Calculation groups let one shared set of time-intelligence expressions apply across all base measures, avoiding the need to duplicate each calculation 50 times over.

Reference: learn.microsoft.com/training/modules/design-scalable-semantic-models/

Question 43 of 50 | Semantic Model Design & Optimization

CORRECT

In Tabular Editor 2, you need to load data into a table without rebuilding hierarchies/relationships or recalculating calculated columns/measures. Which process mode should you use?

Your Answer (correct): Process Data

Justification: Process Data refreshes only the table's data, leaving hierarchies, relationships, calculated columns, and measures untouched. Process Default rebuilds/recalculates those dependent objects, which is exactly what should be avoided here.

Reference:

learn.microsoft.com/analysis-services/tabular-models/process-database-table-or-partition-analysis-services

Question 44 of 50 | Semantic Model Design & Optimization

INCORRECT

dimCustomer must report historical sales by the customer's zip code at the time of sale (not their current zip code), using a fixed number of columns. Which SCD type should you choose?

Your Answer: type 0 slowly changing dimension (SCD)

Correct Answer: type 2 slowly changing dimension (SCD)

Justification: Type 2 SCD preserves history by adding new rows (with new keys) whenever an attribute changes, enabling accurate point-in-time reporting. Type 0 assumes attributes never change at all, which doesn't fit a value (zip code) that can change over time.

Reference: learn.microsoft.com/training/modules/load-data-into-microsoft-fabric-data-warehouse/2-explore-data-load-strategies

Question 45 of 50 | Semantic Model Design & Optimization

CORRECT

You need to audit a semantic model for performance/design issues you own. Which Tabular Editor tool should you use?

Your Answer (correct): Best Practices Analyzer

Justification: Best Practices Analyzer applies a configurable ruleset to flag issues and offers quick-fix scripts, uniquely suited to a design/performance audit.

Reference: learn.microsoft.com/power-bi/transform-model/desktop-external-tools

Question 46 of 50 | Semantic Model Design & Optimization

INCORRECT

A slow DAX measure filters on Sales[Year]. You need the best-performing version of the filter. Which measure should you use?

Your Answer: CALCULATE([Sales], Sales[Year] = 2023)

Correct Answer: CALCULATE([Sales], Calendar[Year] = 2023)

Justification: Filtering on the (typically much smaller) dimension table — here Calendar[Year] — is faster than filtering directly on the large fact table, since it reduces the work done by both the formula and storage engines.

Reference: learn.microsoft.com/dax/best-practices/dax-avoid-avoid-filter-as-filter-argument

Question 47 of 50 | Data Ingestion & Transformation

INCORRECT

A managed Delta table has 1,000 small (1 MB) Parquet files and slow queries; 7-day change history retention is required. What should you do from Lakehouse explorer's Maintenance options?

Your Answer: Run VACUUM with a 7-day retention policy only.

Correct Answer: Run OPTIMIZE as well as VACUUM with a 7-day retention policy.

Justification: OPTIMIZE compacts many small files into fewer, larger ones (ideal size ~128 MB–1 GB), directly fixing the query performance problem; VACUUM alone only removes old unused files and doesn't reduce the number of active files being scanned.

Reference: learn.microsoft.com/fabric/data-engineering/lakehouse-table-maintenance

Question 48 of 50 | Data Ingestion & Transformation

CORRECT

You need lakehouse tables optimized for Direct Lake connections used by semantic models. What should you do?

Your Answer (correct): Use OPTIMIZE to apply V-Order.

Justification: V-Order optimization (applied via OPTIMIZE) restructures Parquet files for significantly faster Direct Lake read performance.

Reference: learn.microsoft.com/fabric/data-engineering/delta-optimization-and-v-order

Question 49 of 50 | Data Ingestion & Transformation

INCORRECT

A Dataflow Gen2 query with many Power Query transformations occasionally times out during ingestion of FactSales, and neither further optimization nor a refresh-time change has helped. What additional action should you recommend?

Your Answer: Create a shortcut to FactSales, connect the dataflow to the shortcut, then apply the transformations.

Correct Answer: Create a second 'staging' dataflow that ingests FactSales with no transformations, then have the original dataflow reference this staging dataflow for the transform step.

Justification: Splitting ingestion (staging dataflow) from transformation (a second dataflow referencing it) is the recommended pattern to resolve timeout/performance issues in complex dataflows. A shortcut to the same table doesn't reduce the transformation workload causing the timeout.

Reference: learn.microsoft.com/power-query/dataflows/best-practices-developing-complex-dataflows

Question 50 of 50 | Semantic Model Design & Optimization

INCORRECT

You need to optimize a DAX measure using nested IFs so it calculates faster. Which code should you use?

Your Answer: VAR Calc = [Variance] RETURN SWITCH(TRUE(), [Variance] > 0.80, "Amazing!", [Variance] > 0.60, "Good", "Bad") — (declares Calc but never uses it)

Correct Answer: VAR Calc = [Variance] RETURN SWITCH(TRUE(), Calc > 0.80, "Amazing!", Calc > 0.60, "Good", "Bad")

Justification: Declaring [Variance] as a variable (Calc) and then actually referencing Calc in each comparison caches the measure's result so it's evaluated once instead of repeatedly — the key performance gain. Declaring the variable but still calling [Variance] directly forfeits that caching benefit.

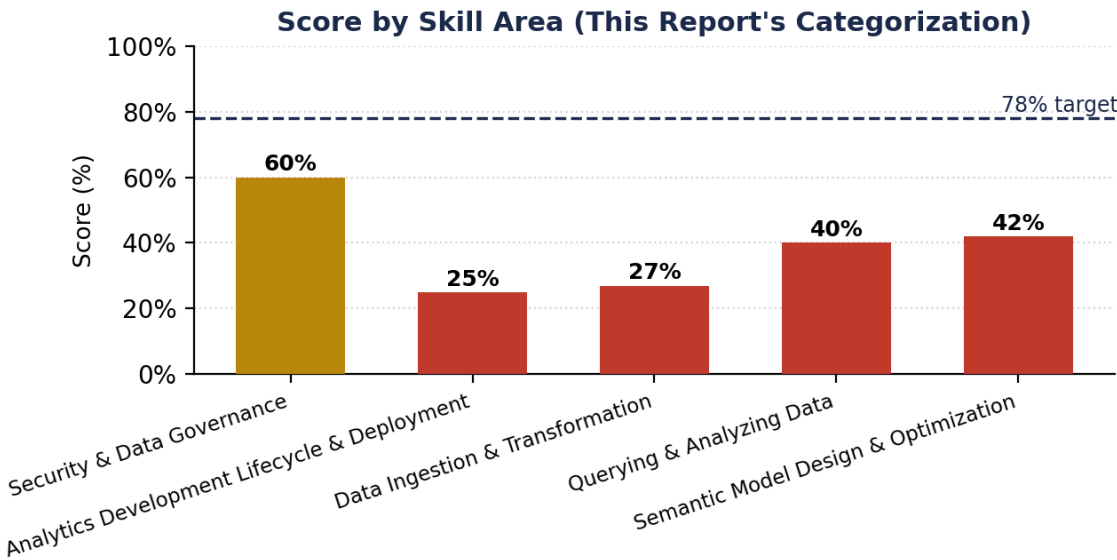
Reference: learn.microsoft.com/dax/best-practices/dax-variables

Performance Summary & Study Report

Overall score: **36%** (18 of 50 questions correct) in 58 minutes. The source assessment recommends aiming for **78% or higher** across multiple attempts before sitting the real DP-600 exam.

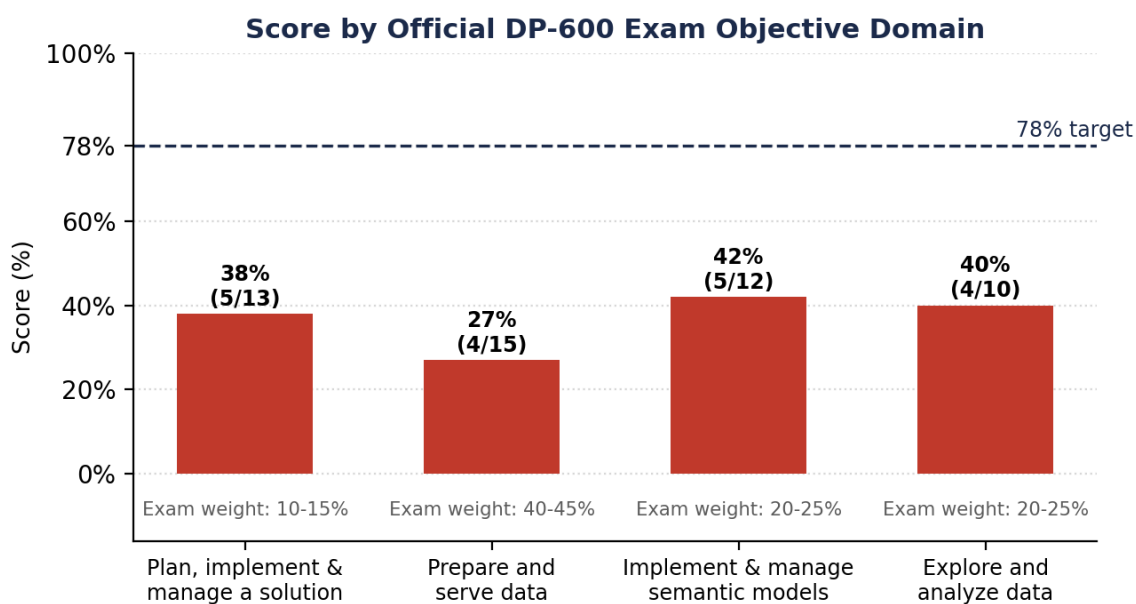
Score by Skill Area

Skill Area	Correct	Total	Score	Status
Security & Data Governance	3	5	60%	Developing
Analytics Development Lifecycle & Deployment	2	8	25%	Needs Focus
Data Ingestion & Transformation	4	15	27%	Needs Focus
Querying & Analyzing Data	4	10	40%	Needs Focus
Semantic Model Design & Optimization	5	12	42%	Needs Focus



Score by Official DP-600 Exam Objective Domain

Microsoft's published DP-600 exam outline groups skills into four objective domains, each carrying a different weight on the real exam. Mapping your results onto these domains shows where extra study time will move your overall score the most:



Key Takeaways

- **Highest-leverage fix:** "Prepare and serve data" carries the largest exam weight (40-45%) and is currently your lowest-scoring objective domain at 27% (4/15) — improving here moves your overall score more than any other domain.
- **Weakest areas:** Analytics Development Lifecycle & Deployment (2/8) and Data Ingestion & Transformation (4/15). These should be the priority for re-study before the next attempt.
- **Strongest area:** Security & Data Governance (3/5) — a solid foundation here; light review should be enough to maintain it.
- **Recurring error pattern in Ingestion & Transformation:** repeated confusion between Copy data activity (no-code, high-throughput, no transforms) vs. Dataflow Gen2 (low-code, transforms, smaller data) vs. notebooks (code, complex transforms, large scale) — review the decision tree in "Options to get data into the Lakehouse."
- **Recurring error pattern in Deployment Lifecycle:** mixing up reusable-asset file types (.pbids vs .pbit vs .pbip) and pipeline/Git mechanics (deployment rules, autobinding, lineage view, branching workspaces, shortcut resolution across stages).
- **Recurring error pattern in Query/DAX:** aggregation order matters — SUMX vs. SUM, DENSE_RANK vs. RANK, HAVING vs. WHERE, and filtering dimension tables rather than fact tables for performance.
- **Recurring error pattern in Model Optimization:** storage-mode combinations for composite models (Import / DirectQuery / Dual), Direct Lake for near-real-time reporting, and SCD Type 2 for point-in-time historical attributes.

Recommended Next Steps

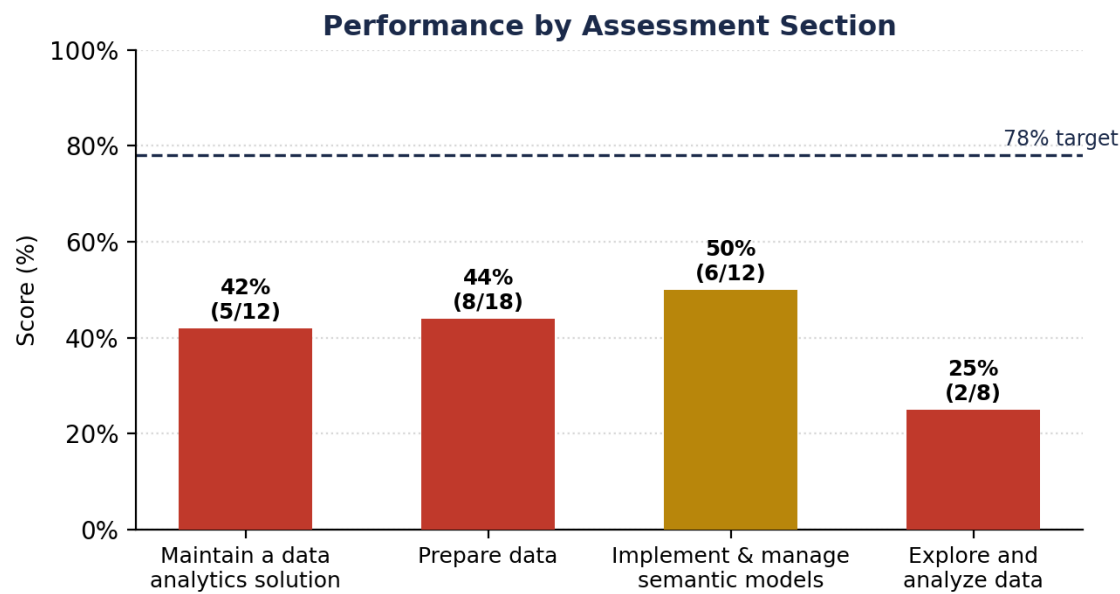
1. Re-read the justification for every question marked INCORRECT above; each includes the reasoning for why the distractor was wrong, not just the right answer.
2. Focus first on Data Ingestion & Transformation and the Analytics Development Lifecycle, since these currently show the lowest scores.
3. Work through the linked Microsoft Learn modules for each missed question, especially the ingestion decision-making module and the CI/CD (deployment pipelines and Git integration) module.
4. Retake a fresh practice assessment after review, and track score-by-area again to confirm improvement before the 78%+ benchmark is reached.

DP-600 Practice Assessment Results

Implementing Analytics Solutions Using Microsoft Fabric

Attempt 2 • September 19, 2026

Score	42% (21 / 50 correct)
Time Taken	32 minutes
Target Benchmark	78% or higher (per source recommendation)
Change vs. Attempt 1	+6 points (36% → 42%)



Question 1 of 50 | Maintain a data analytics solution**INCORRECT**

User1 needs read-only SQL access to a lakehouse only, no access to the rest of the workspace, and no Spark access to files. What should you do?

Your Answer: Add User1 to the workspace as a Viewer, share the lakehouse with User1, and select 'Read all SQL Endpoint data.'

Correct Answer: Share the lakehouse with User1 directly and select 'Read all SQL Endpoint data.' (do NOT add as a workspace member)

Justification: Adding User1 to the workspace as any role — even Viewer — grants access to every other item in the workspace, including all Power BI reports, which violates the requirement. Sharing the lakehouse directly (without workspace membership) is sufficient and correctly scoped.

Reference: learn.microsoft.com/fabric/data-engineering/lakehouse-sharing

Question 2 of 50 | Prepare data**INCORRECT**

Lakehouse1 has FactSales partitioned by CustomerID. You need a shortcut in Lakehouse2 that only connects to CustomerID 100's data. What should you do?

Your Answer: In the connected semantic models, add a report-level filter for CustomerKey = 100.

Correct Answer: While creating the shortcut, select the CustomerKey=100 folder under the FactSales folder in Tables.

Justification: During shortcut creation you can expand the partitioned folder structure and point the shortcut directly at the CustomerID=100 partition folder — so only that data is ever connected. A report-level filter still connects to (and can expose) all customer data; it doesn't limit what the shortcut itself accesses.

Reference: learn.microsoft.com/fabric/data-engineering/lakehouse-shortcuts

Question 3 of 50 | Maintain a data analytics solution**CORRECT**

Which three actions restrict access to sensitive financial data based on role while complying with regulatory standards?

Your Answer (correct): Apply sensitivity labels; Implement row-level security; Use dynamic data masking.

Justification: Sensitivity labels classify and protect data for compliance, row-level security restricts access by role, and dynamic data masking hides sensitive values while still allowing analysis. Endorsing the warehouse only signals trust and a blanket Viewer role doesn't provide the needed access granularity.

Reference: learn.microsoft.com/training/modules/secure-data-warehouse-in-microsoft-fabric/

Question 4 of 50 | Maintain a data analytics solution**INCORRECT**

Which three actions safeguard sensitive data at different levels within a data warehouse?

Your Answer: Apply sensitivity labels; Configure object-level security; Use role-based access control.

Correct Answer: Apply sensitivity labels; Configure workspace roles; Configure object-level security.

Justification: Workspace roles plus object-level security plus sensitivity labels together cover access at the workspace, object, and classification levels. Generic 'role-based access control' isn't a specific Fabric mechanism here and lacks the granularity workspace roles and object-level security provide.

Reference: learn.microsoft.com/training/modules/secure-data-warehouse-in-microsoft-fabric/

Question 5 of 50 | Maintain a data analytics solution**CORRECT**

Which two actions protect sensitive data within a Fabric lakehouse from unauthorized access?

Your Answer (correct): Apply sensitivity labels; Implement row-level security.

Justification: Sensitivity labels classify and protect information for compliance, while row-level security restricts data access based on user role — together providing the needed granularity. Workspace roles and item-level permissions alone aren't granular enough to protect specific sensitive data.

Reference: learn.microsoft.com/training/modules/secure-data-access-in-fabric/

Question 6 of 50 | Maintain a data analytics solution**CORRECT**

You need to control access to data within a report based on user roles. What should you configure?

Your Answer (correct): Configure row-level security.

Justification: Row-level security filters which rows a user can see based on role, providing the necessary granularity. Workspace roles are too broad, sensitivity labels address compliance rather than filtering, and item-level access controls the whole report, not the data within it.

Reference: learn.microsoft.com/training/modules/administer-fabric/4-manage-security

Question 7 of 50 | Maintain a data analytics solution**CORRECT**

Users of an external tool cannot deploy metadata changes to SalesModel in Sales-Prod. What should you configure?

Your Answer (correct): Set the XMLA endpoint to Read Write on the capacity that hosts the workspace.

Justification: External tools need Read Write XMLA access to deploy metadata changes to a semantic model. Downloading/republishing a PBIX disrupts the development lifecycle and isn't required; tenant item-creation settings and Admin permissions alone don't grant write access without the XMLA setting itself.

Reference: learn.microsoft.com/fabric/enterprise/powerbi/troubleshoot-xml-analysis-endpoint

Question 8 of 50 | Maintain a data analytics solution**INCORRECT**

You connect Workspace1 to Git (Repo1). You need multiple users to collaborate on the semantic model and report with minimal development effort. What should you do?

Your Answer: Create a new workspace and connect the new workspace to a local folder.

Correct Answer: From Workspace1, create a branch and a new workspace.

Justification: Fabric Git integration lets users branch into their own mapped workspace to work independently, then merge back — the built-in collaboration pattern. Fabric does not support connecting a workspace directly to a local folder at all.

Reference: learn.microsoft.com/training/modules/implement-cicd-in-fabric/3-implement-version-control-and-git-integration

Question 9 of 50 | Maintain a data analytics solution**INCORRECT**

Authors need a reusable report template with no embedded data, branding, preset slicer placement, and a standard layout. What should you create?

Your Answer: a Power BI data source file (.pbids)

Correct Answer: a Power BI template file (.pbit)

Justification: A .pbit packages report layout, visuals, branding, and configuration with no embedded data — exactly a distributable template. A .pbids only stores a data-source connection; it defines no report layout at all.

Reference: learn.microsoft.com/power-bi/guidance/fabric-adoption-roadmap-mentoring-and-user-enablement

Question 10 of 50 | Maintain a data analytics solution

INCORRECT

SalesLH has an internal OneLake shortcut to another lakehouse table in the same workspace. You deploy SalesLH to Test via a pipeline. What should you do?

Your Answer: Enable Git integration for the workspace so lakehouse items are tracked in source control.

Correct Answer: Create the tables, folders, and files referenced by the shortcut in the target workspace before the deployment.

Justification: Internal OneLake shortcuts require the referenced object to already exist in the destination workspace, so it must be created there before deployment. Git integration tracks source control changes but does not resolve missing shortcut targets during a deployment pipeline run.

Reference: learn.microsoft.com/fabric/data-engineering/lakehouse-git-deployment-pipelines

Question 11 of 50 | Maintain a data analytics solution

CORRECT

Sales-Dev (West US) needs to connect to an Azure DevOps Git repo in West Europe. What should you do?

Your Answer (correct): Enable the tenant setting that allows exporting items to Git repositories in other geographical locations.

Justification: Cross-region Git connections require this specific tenant setting to be enabled. Workspace regions can't be changed after creation, outbound protection settings don't govern Git integration, and workspace identity is for authentication, not cross-region connectivity.

Reference: learn.microsoft.com/fabric/cicd/troubleshoot-cicd

Question 12 of 50 | Maintain a data analytics solution

INCORRECT

SalesModel (shared, used by multiple reports) will have a column renamed. You need to identify downstream reports/workspaces that might be affected. What should you use?

Your Answer: The SQL analytics endpoint

Correct Answer: data lineage view

Justification: The data lineage view visually maps upstream/downstream dependencies, showing which reports and workspaces rely on a semantic model before you make a breaking change. The SQL analytics endpoint is for querying data, not for tracing dependencies.

Reference: learn.microsoft.com/fabric/data-science/semantic-model-best-practices

Question 13 of 50 | Maintain a data analytics solution

INCORRECT

You plan to share a semantic model to another workspace with five more reports. From the open semantic model, what should you select next to analyze downstream dependencies with minimal admin effort?

Your Answer: Select Security.

Correct Answer: Select Impact analysis.

Justification: Impact analysis automatically summarizes all downstream dependencies (reports and cross-workspace usage) with minimal effort. Security only governs permissions and has nothing to do with dependency tracking.

Reference: learn.microsoft.com/fabric/governance/impact-analysis

Question 14 of 50 | Prepare data

CORRECT

You need code-free data ingestion into a Fabric warehouse with the ability to apply custom transformations before loading. What should you use?

Your Answer (correct): Dataflows Gen2

Justification: Dataflows Gen2 (Power Query) supports no-code ingestion plus custom transformations before load. Power Automate isn't built for this, and Data Factory/notebooks typically require code for custom transforms.

Reference: learn.microsoft.com/training/modules/use-data-factory-pipelines-fabric/3-copy-data

Question 15 of 50 | Prepare data

CORRECT

You create a data pipeline to ingest data into Lakehouse1 using the Copy data activity. Which General-tab properties are mandatory?

Your Answer (correct): Name only

Justification: For the Copy Data activity, the Name field on the General tab is the only mandatory property — everything else there is optional.

Reference: learn.microsoft.com/fabric/data-engineering/tutorial-lakehouse-data-ingestion

Question 16 of 50 | Prepare data

CORRECT

You need to combine sales data in Warehouse2 with HR data in Warehouse1, minimizing development effort and data movement. What should you recommend?

Your Answer (correct): Use cross-database querying between Warehouse1 and Warehouse2.

Justification: Cross-database querying lets you query across Fabric warehouses directly via the SQL analytics endpoint with no data movement. Copying data between warehouses would work but unnecessarily duplicates data.

Reference: learn.microsoft.com/training/modules/get-started-data-warehouse/3-understand-data-warehouse-fabric

Question 17 of 50 | Prepare data

INCORRECT

You need to ingest a 1 TB external dataset into a lakehouse with the highest throughput, using a low-code/no-code option. What should you recommend?

Your Answer: Use Dataflow Gen2 to import the files and load them to a table.

Correct Answer: Use the Copy data activity of a pipeline to copy the data.

Justification: The Copy data activity is both low-code/no-code and optimized for high-throughput bulk transfer of large datasets. Dataflow Gen2 is not as efficient as Copy data activity for a straight bulk copy at this scale.

Reference: learn.microsoft.com/fabric/data-engineering/load-data-lakehouse

Question 18 of 50 | Prepare data

INCORRECT

Pipeline1 runs in the US-West region. You need it to run at midnight in the UTC-0 time zone. How should you configure the schedule?

Your Answer: Add a data pipeline notebook activity to convert the US West time zone to UTC-0.

Correct Answer: For Pipeline1, set the scheduler time zone to UTC-0.

Justification: Each data pipeline has its own scheduler time zone setting that applies directly to that pipeline's trigger — no conversion logic is needed inside the pipeline itself.

Reference: learn.microsoft.com/fabric/data-factory/pipeline-runs

Question 19 of 50 | Prepare data

INCORRECT

Raw data is staged then transformed into final lakehouse tables. What is the most efficient way to run this ETL process?

Your Answer: Use Dataflows Gen2.

Correct Answer: Use Fabric notebooks.

Justification: Notebooks run on Apache Spark, giving the scale and processing power needed for large-dataset ETL inside a lakehouse. Dataflow Gen2/Power Query is useful but less efficient than Spark at this scale.

Reference: learn.microsoft.com/training/modules/get-started-lakehouses/3-work-lakehouse

Question 20 of 50 | Prepare data**CORRECT**

Notebook1 ingests data, Notebook2 transforms it; Pipeline1 runs them in sequence. You need the process to run daily at 7:00 AM. On which object should you configure the schedule?

Your Answer (correct): Pipeline1

Justification: Pipeline-level scheduling runs the full orchestrated sequence (Notebook1 then Notebook2) reliably. Scheduling an individual notebook wouldn't guarantee the two run in the correct order together.

Reference: learn.microsoft.com/fabric/data-factory/pipeline-runs

Question 21 of 50 | Prepare data**CORRECT**

You need to ingest already-formatted forecast data from ADLS Gen2 into a lakehouse with no further transformation, minimizing dev effort and cost. What should you recommend?

Your Answer (correct): Use the Copy activity in a pipeline.

Justification: When no transformation is needed, Copy data activity moves data directly and efficiently between source and destination — no Dataflow Gen2 or notebook overhead required.

Reference: learn.microsoft.com/training/modules/use-data-factory-pipelines-fabric/3-copy-data

Question 22 of 50 | Prepare data**INCORRECT**

You need to ingest a 1 TB dataset into Warehouse1 with the highest throughput and a low-code/no-code model. What should you recommend?

Your Answer: Dataflow Gen2

Correct Answer: Copy data activity

Justification: Copy data activity gives the highest throughput for large ingestions with no transformation while still being low-code/no-code. Dataflow Gen2 is stronger for transformation scenarios, not raw bulk ingestion at this scale.

Reference: learn.microsoft.com/fabric/data-warehouse/ingest-data

Question 23 of 50 | Prepare data**INCORRECT**

1 TB of legacy accounting data (queried rarely, very selectively) sits in ADLS Gen2. You also need to ingest sales data for regular reporting and support year-end ad-hoc queries on the legacy data, minimizing admin effort/cost. What should you recommend?

Your Answer: Ingest the sales data into the Fabric warehouse and use a pipeline to move the legacy accounting data into the warehouse.

Correct Answer: Ingest the sales data into the Fabric lakehouse and set up a shortcut to the legacy accounting data in the storage account.

Justification: Since the legacy data is rarely and selectively queried, a OneLake shortcut avoids unnecessary data movement and cost. Moving the legacy data into the warehouse via a pipeline creates ongoing storage and maintenance overhead that isn't needed for such infrequent access.

Reference: learn.microsoft.com/fabric/onelake/onelake-shortcuts

Question 24 of 50 | Prepare data**INCORRECT**

UnpostedSales is truncated and replaced daily in the source. You ingest it via Dataflow Gen2 and must retain all current AND historical records. What should you do?

Your Answer: Use OPTIMIZE to apply V-order for the query.

Correct Answer: Configure the refresh to append data for the query.

Justification: Appending on refresh adds each day's new rows to what's already stored, preserving both historical and current records. V-Order only optimizes file layout for read performance — it does nothing to control whether old records are kept or overwritten.

Reference: learn.microsoft.com/fabric/data-engineering/load-to-tables

Question 25 of 50 | Explore and analyze data**CORRECT**

In a notebook, you need the total record count per ProductKey, sorted descending. Which statement should you run?

Your Answer (correct): `df.groupBy("ProductKey").count().sort("count", descending=True).show()`

Justification: `groupBy` groups rows by `ProductKey`, `count()` tallies records per group, `sort(..., descending=True)` orders the result, and `show()` displays it — exactly the requested query.

Reference: learn.microsoft.com/training/modules/use-apache-spark-work-files-lakehouse/

Question 26 of 50 | Prepare data**INCORRECT**

You need PySpark code to save DataFrame df (with Year/Quarter columns) as a Delta table named Sales, partitioned by Year and Quarter. Which command should you run next?

Your Answer:

```
df.write.mode("overwrite").format("delta").partitionBy("Year",Quarter").save("Files/Sales")
```

Correct Answer:

```
df.write.mode("overwrite").format("delta").partitionBy("Year","Quarter").save("Tables/Sales")
```

Justification: Saving to Tables/ (not Files/) is what registers the output as a queryable Delta table in the lakehouse; saving under Files/ would leave it as unmanaged file data, not a table.

Reference: learn.microsoft.com/fabric/data-engineering/tutorial-lakehouse-data-preparation

Question 27 of 50 | Prepare data**CORRECT**

A 'Product ID/Name' column concatenates two values. You need one applied step that both removes the original column and creates two new columns. Which applied step should you use?

Your Answer (correct): Split Column

Justification: Split Column is the only Power Query step that removes the source column and creates the resulting new columns in a single applied step.

Reference: learn.microsoft.com/power-query/split-columns-delimiter

Question 28 of 50 | Prepare data**CORRECT**

You need a new column labeling products 'High' if unit price > \$1,000 and 'Regular' otherwise. What should you select on the Add column tab?

Your Answer (correct): Conditional column

Justification: Conditional column lets you build a new column whose values are derived from one or more conditions applied to existing columns — exactly this High/Regular split.

Reference: learn.microsoft.com/power-query/add-conditional-column

Question 29 of 50 | Explore and analyze data**CORRECT**

In Power Query Editor, you need to identify the percentage of valid records in a column before loading to a report. Which option should you use?

Your Answer (correct): Column quality

Justification: Column quality shows the percentage of valid, error, and empty values in a column — column distribution shows frequency, and column profile shows broader statistics, but neither gives a direct 'percent valid' figure.

Reference: learn.microsoft.com/training/modules/clean-data-power-bi/6-profile-data

Question 30 of 50 | Prepare data**CORRECT**

You use Dataflow Gen2 to ingest and transform data from Azure SQL Database into a lakehouse. Which language should you use to transform the data?

Your Answer (correct): M

Justification: Dataflow Gen2 uses the same Power Query engine regardless of data source, and Power Query's transformation language is always M.

Reference: learn.microsoft.com/power-query/power-query-what-is-power-query

Question 31 of 50 | Explore and analyze data**INCORRECT**

Using a Sales table, you need the total number of products sold for each Product_ID in January 2024. What should you run?

Your Answer: SELECT Product_ID, COUNT(Quantity) FROM Sales WHERE Sales_Date LIKE '2023-01-%' GROUP BY Product_ID

Correct Answer: SELECT Product_ID, SUM(Quantity) FROM Sales WHERE MONTH(Sales_Date)=1 AND YEAR(Sales_Date)=2024 GROUP BY Product_ID

Justification: SUM(Quantity) totals the units actually sold, not COUNT(Quantity) which only counts rows. The date filter must also target January 2024 specifically — the submitted answer both used the wrong aggregate and filtered on the wrong year (2023) via an unreliable string pattern.

Reference: learn.microsoft.com/sql/t-sql/queries/where-transact-sql

Question 32 of 50 | Explore and analyze data**INCORRECT**

You need a DAX measure 'Total Profit' that computes profit per transaction line and aggregates correctly when sliced by product/date. Which expression should you use?

Your Answer: Total Profit = SUM(Sales[Quantity]) * (SUM(Sales[Price]) - SUM(Sales[Cost]))

Correct Answer: Total Profit = SUMX(Sales, Sales[Quantity] * (Sales[Price] - Sales[Cost]))

Justification: SUMX iterates row by row, computing profit per transaction line before summing — required because quantity, price, and cost vary per row. Multiplying pre-aggregated SUMs produces mathematically incorrect totals whenever these values differ across rows.

Reference: learn.microsoft.com/training/modules/design-scalable-semantic-models/5-dax-optimizations

Question 33 of 50 | Explore and analyze data**INCORRECT**

In the visual Query Editor, you need to summarize passenger counts by year, joining Trip and Date. Which transformations should you use?

Your Answer: Choose GroupBy on Trip, then use Append queries to join summarized Trip data and Date.

Correct Answer: Choose Merge queries to join Trip and Date, then use GroupBy.

Justification: Merge performs the actual join between Trip and Date (via expanding matched columns), after which GroupBy can summarize by year. Append performs a union, not a join, so it can't combine the two tables' columns.

Reference: learn.microsoft.com/fabric/data-warehouse/tutorial-visual-query

Question 34 of 50 | Explore and analyze data**INCORRECT**

You created an inner-join merge of FactSales and DimCustomer in the visual query editor. You need View query to generate T-SQL including the join and aggregations. What should you do first?

Your Answer: Save the query as a view and view the SQL from the saved view.

Correct Answer: Enable load for the merged query result.

Justification: Enabling load materializes the merged query in the visual pipeline so View query can generate matching T-SQL, including the join and aggregation steps. Saving as a view happens after SQL generation, not before, and doesn't itself trigger the T-SQL view.

Reference: learn.microsoft.com/fabric/data-warehouse/visual-query-editor

Question 35 of 50 | Explore and analyze data**INCORRECT**

You need to analyze recent storm event data in a KQL database using Kusto Query Language. What should you do?

Your Answer: Use a KQL activity in a data pipeline.

Correct Answer: Use a KQL Queryset.

Justification: A KQL Queryset is the direct, purpose-built tool for writing and running KQL queries against a KQL database. A pipeline KQL activity is meant for orchestration, not interactive querying.

Reference: learn.microsoft.com/training/modules/get-started-lakehouses/3-work-lakehouse

Question 36 of 50 | Explore and analyze data

INCORRECT

Using FactSales and DimProduct, you need a monthly report of sales by product category via the visual query editor. Which two actions should you take?

Your Answer: Merge FactSales and DimProduct on ProductKey; Sort products by sales amount.

Correct Answer: Add FactSales and DimProduct tables (to the canvas); Merge FactSales and DimProduct on ProductKey.

Justification: Before any merge can happen, both tables must first be added to the visual query editor's canvas — that step can't be skipped. Sorting isn't a necessary step for building this particular summarized report.

Reference:

learn.microsoft.com/training/modules/query-data-warehouse-microsoft-fabric/4-explore-visual-query-editor

Question 37 of 50 | Implement and manage semantic models

INCORRECT

Two unrelated tables in a model need a virtual relationship. Which DAX function should you use?

Your Answer: USERELATIONSHIP()

Correct Answer: TREATAS()

Justification: TREATAS() applies the results of a table expression as filters onto columns in an otherwise unrelated table, creating a virtual relationship. USERELATIONSHIP() only activates an existing (already-defined) inactive physical relationship — it can't create a relationship where none exists.

Reference: learn.microsoft.com/dax/treatas-function

Question 38 of 50 | Implement and manage semantic models

INCORRECT

On F64, a lakehouse has 1B historical records plus ~10,000 near-real-time updates every 15 minutes. You need the best report performance while supporting near-real-time reporting. Which storage mode should you use?

Your Answer: Import and Direct Lake combined

Correct Answer: Direct Lake

Justification: Direct Lake reads data directly from OneLake, giving near-real-time freshness with performance close to Import — without Import's memory-loading delay or DirectQuery's slower query pattern. Direct Lake tables cannot currently be mixed with Import/DirectQuery/Dual in the same model, so a 'combined' mode isn't a valid option.

Reference: learn.microsoft.com/power-bi/enterprise/directlake-overview

Question 39 of 50 | Implement and manage semantic models

CORRECT

You need the same time-intelligence calculations (MTD, YTD, YoY) applied across 50 different measures, minimizing admin effort. What should you do?

Your Answer (correct): Create a calculation group.

Justification: Calculation groups let one shared set of time-intelligence expressions apply across all base measures, avoiding the need to duplicate each calculation 50 times over.

Reference: learn.microsoft.com/training/modules/design-scalable-semantic-models/

Question 40 of 50 | Implement and manage semantic models

CORRECT

Sales connects to Date via active (OrderDateKey) and inactive (ShipDateKey) relationships. You need Sales Ordered and Sales Shipped measures analyzable by Year. How should you create them?

Your Answer (correct): Sales Ordered = SUM(Sales[SalesAmount]); Sales Shipped = CALCULATE(SUM(Sales[SalesAmount]), USERELATIONSHIP('Date'[DateKey], Sales[ShipDateKey]))

Justification: Sales Ordered can use a plain SUM since it relies on the active relationship. Sales Shipped needs USERELATIONSHIP to temporarily activate the existing inactive relationship (Date to ShipDateKey) for that one calculation.

Reference: learn.microsoft.com/power-bi/guidance/relationships-active-inactive

Question 41 of 50 | Implement and manage semantic models

CORRECT

dimCustomer must report historical sales by the customer's zip code at the time of sale (not their current zip code), using a fixed number of columns. Which SCD type should you choose?

Your Answer (correct): type 2 slowly changing dimension (SCD)

Justification: Type 2 SCD preserves history by adding new rows (with new keys) whenever an attribute changes, enabling accurate point-in-time reporting without adding extra columns — unlike Type 1 (overwrites history) or Type 3 (adds columns).

Reference: learn.microsoft.com/training/modules/load-data-into-microsoft-fabric-data-warehouse/2-explore-data-load-strategies

Question 42 of 50 | Implement and manage semantic models

INCORRECT

As part of a model audit, you review: `RANKX(ALL('Product'[Product Name]), [Sales],, DESC, Skip)`. What does this measure calculate?

Your Answer: Ranks product names by Sales, largest values get smallest ranks, and tied names get the same rank number (no further detail on what follows a tie).

Correct Answer: Ranks product names by Sales, largest values get smallest ranks, and after a tie the next rank equals the tied rank plus the count of tied items (standard RANK-style, gapped, tie handling).

Justification: The 'Skip' parameter in RANKX specifically produces gapped ranking after ties — e.g., two items tied at rank 1 are followed by rank 3, not rank 2 — because the next rank accounts for how many items shared the tie. A description that stops at 'tied items get the same rank' omits this specific tie-handling behavior that 'Skip' introduces.

Reference: learn.microsoft.com/dax/rankx-function-dax

Question 43 of 50 | Implement and manage semantic models

CORRECT

You need a star schema supporting filtering/grouping by product, customer, store, and date. Which table should you configure as the fact table?

Your Answer (correct): SalesTransactions

Justification: SalesTransactions holds the transactional events and additive measures (Quantity, SalesAmount) plus foreign keys to the descriptive dimensions — the defining traits of a fact table, while Customer/Date/Product/Store hold descriptive attributes as dimensions.

Reference: learn.microsoft.com/fabric/data-warehouse/dimensional-modeling-overview

Question 44 of 50 | Implement and manage semantic models

CORRECT

Report visuals on a lakehouse-based semantic model must always reflect the latest data. Which storage mode should you use?

Your Answer (correct): Direct Lake

Justification: Direct Lake reads Delta tables directly from OneLake with no import or scheduled refresh needed, so visuals always reflect the latest lakehouse data while retaining strong query performance.

Reference: learn.microsoft.com/training/modules/design-scalable-semantic-models/2-storage-modes

Question 45 of 50 | Implement and manage semantic models

INCORRECT

A report uses DirectQuery to Azure SQL. You need to reduce the number of queries sent to the database while users interact with filters/slicers. What should you do?

Your Answer: Change default visual interaction from cross highlighting to cross filtering.

Correct Answer: Add apply buttons to all the basic filters.

Justification: Apply buttons pause all query requests until the user finalizes their filter/slicer selections, then send a single consolidated request — directly reducing query volume. Changing the default cross-interaction behavior doesn't change how many queries are fired per selection.

Reference: learn.microsoft.com/power-bi/create-reports/buttons-apply-all-clear-all-slicers

Question 46 of 50 | Implement and manage semantic models

INCORRECT

A report page renders slowly. You need to identify which element consumes the most rendering time, minimizing admin effort and capture time. What should you use?

Your Answer: DAX Studio

Correct Answer: Performance analyzer

Justification: Performance Analyzer is a built-in Power BI Desktop feature that captures per-element rendering time directly, with no extra setup. DAX Studio analyzes DAX query performance specifically, not the rendering time of each visual/report element.

Reference: learn.microsoft.com/power-bi/create-reports/desktop-performance-analyzer

Question 47 of 50 | Implement and manage semantic models

CORRECT

You need to audit a semantic model you own for performance/design issues. Which Tabular Editor tool should you use?

Your Answer (correct): Best Practices Analyzer

Justification: Best Practices Analyzer applies a configurable ruleset to flag issues and offers quick-fix scripts — uniquely suited to a design/performance audit among Tabular Editor's tools.

Reference: learn.microsoft.com/power-bi/transform-model/desktop-external-tools

Question 48 of 50 | Implement and manage semantic models

INCORRECT

A DAX measure filtering on Sales[Year] runs slowly. Which measure gives the best performance?

Your Answer: CALCULATE([Sales], FILTER(Sales, Sales[Year] = 2023))

Correct Answer: CALCULATE([Sales], Calendar[Year] = 2023)

Justification: Filtering directly on the (typically much smaller) Calendar dimension table is faster than filtering on the large fact table, and a simple boolean filter argument avoids the extra overhead FILTER() introduces by scanning the fact table row by row.

Reference: learn.microsoft.com/dax/best-practices/dax-avoid-avoid-filter-as-filter-argument

Question 49 of 50 | Prepare data

INCORRECT

A managed Delta table has 1,000 small (1 MB) Parquet files and slow queries; 7-day change history retention is required. What should you do from Lakehouse explorer?

Your Answer: Select Maintenance and run the VACUUM command with a retention policy of seven days.

Correct Answer: Select Maintenance and run the OPTIMIZE command as well as the VACUUM command with a retention policy of seven days.

Justification: OPTIMIZE compacts many small files into fewer, larger ones (ideal size ~128 MB–1 GB), directly fixing the query performance problem; VACUUM alone only removes old unused files and doesn't reduce the number of active files scanned during a query.

Reference: learn.microsoft.com/fabric/data-engineering/lakehouse-table-maintenance

Question 50 of 50 | Prepare data

INCORRECT

A Dataflow Gen2 query splits a concatenated OrderDateTime column by position, then filters the split result to the current year, and runs slowly against 20 years of source data. What should you recommend to improve performance?

Your Answer: Insert a new step to force the table to load in full by using Table.Buffer.

Correct Answer: Reorder step 3 so the current-year filter is applied to OrderDateTime first, then apply the split-by-position transformation.

Justification: Splitting a column by position breaks query folding, so filtering only after the split forces all 20 years of data to load into Power Query before the current-year filter ever applies. Filtering first (which can fold back to the source) then splitting keeps folding intact; Table.Buffer would instead force an early full load, making performance worse, not better.

Reference: learn.microsoft.com/power-query/power-query-folding

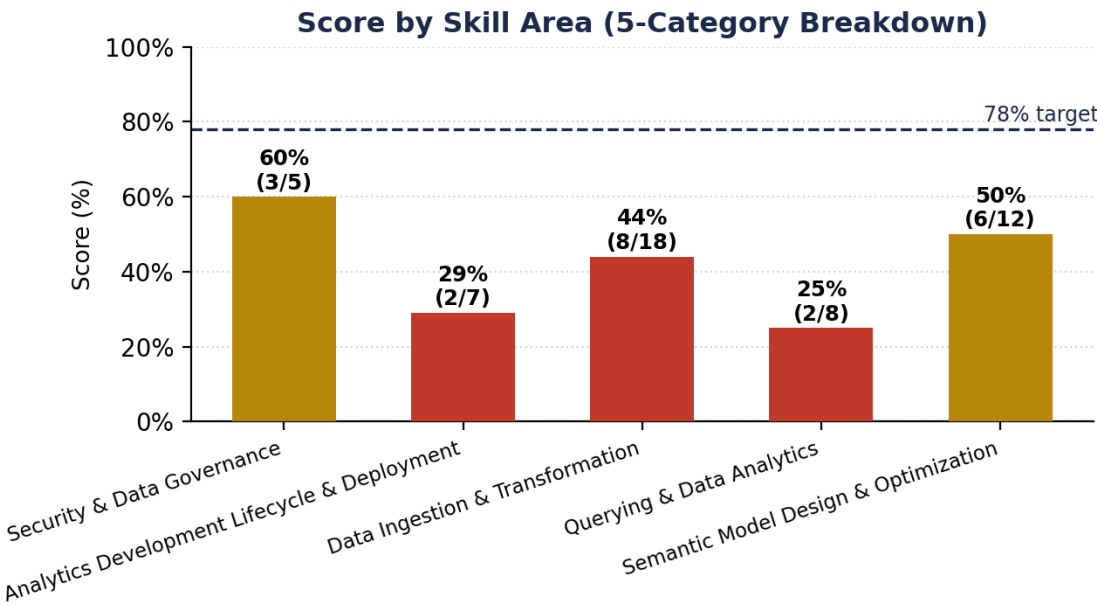
Performance Summary & Study Report

Overall score: **42%** (21 of 50 correct) in 32 minutes — up from 36% on the first attempt. The source assessment recommends aiming for **78% or higher** across multiple attempts before sitting the real DP-600 exam.

Assessment Section	Correct	Total	Score	Status
Maintain a data analytics solution	5	12	42%	Needs Focus
Prepare data	8	18	44%	Needs Focus
Implement and manage semantic models	6	12	50%	Developing
Explore and analyze data	2	8	25%	Needs Focus

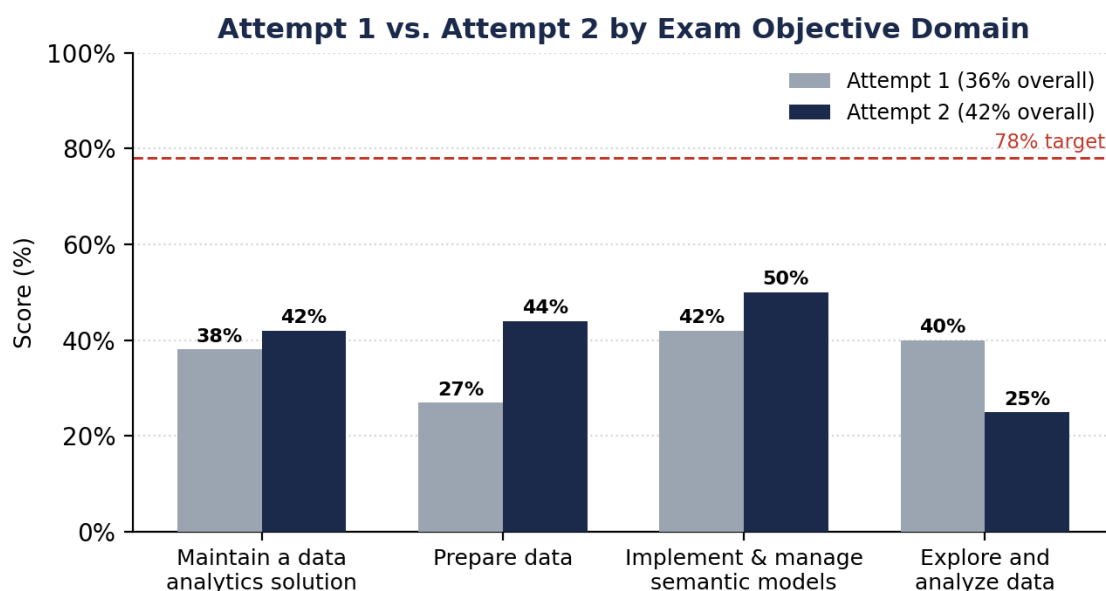
Score by Skill Area (Detailed 5-Category View)

Splitting the official sections further into five more specific skill areas — Security & Data Governance, Analytics Development Lifecycle & Deployment, Data Ingestion & Transformation, Querying & Data Analytics, and Semantic Model Design & Optimization — pinpoints exactly which sub-skills need work within each larger domain:



Progress Since Attempt 1

Comparing this attempt to the first practice assessment (36% overall) by exam objective domain shows where the extra study time paid off, and where the same gaps remain:



Key Takeaways

- **Overall improvement:** score rose from 36% to 42% (+6 points) while time spent dropped from 58 to 32 minutes — a genuine efficiency and accuracy gain, not just a faster guess rate.
- **Now weakest:** Explore and analyze data (2/8, 25%). This domain (SQL, DAX querying, the visual query editor, and KQL) needs the most attention next — six of eight misses here came from picking a plausible-sounding but incomplete or slightly-off answer (wrong aggregate function, wrong applied-step order, or missing a setup step) rather than a completely wrong concept.
- **Still your highest-leverage fix:** "Prepare data" carries the largest exam weight (40-45%) and sits at 44% (8/18) — the same Copy data activity vs. Dataflow Gen2 vs. notebooks decision tree from Attempt 1 is still the main source of errors.
- **Strongest domain:** Implement and manage semantic models (6/12) — semantic model design fundamentals (star schema, SCD Type 2, calculation groups, Direct Lake, role-playing dimensions) are solid; the remaining misses are concentrated in DAX/model *performance* optimization (Performance Analyzer, filtering on dimension vs. fact tables, TREATAS) rather than design.
- **Recurring error pattern — near-miss answers:** several misses this round were not wrong concepts but incomplete correct ones — e.g., listing sensitivity labels + object-level security but forgetting workspace roles, or merging tables without first adding them to the canvas. Read every multi-part question for the FULL required action set before answering.
- **Query folding awareness:** the Dataflow Gen2 step-ordering question (place filters before transformations that break folding, such as Split Column) is a distinct, testable skill — review "Query folding" in Power Query documentation directly.
- **Sharpest sub-skill gap:** in the 5-category view, Analytics Development Lifecycle & Deployment sits lowest at 29% (2/7) — Git integration, deployment pipelines, XMLA endpoints, and impact analysis/lineage tooling are the specific mechanics to revisit, even though the broader "Maintain a data analytics solution" section (which also includes security, where you scored 60%) looks less severe at 42% overall.

Recommended Next Steps

1. Re-read the justification for every question marked INCORRECT above, paying special attention to questions you got "almost" right — those reveal specific missing steps rather than misunderstood concepts.

2. Prioritize "Explore and analyze data" (now your lowest section) and "Prepare data" (highest exam weight) in that order for the next study block.
3. Complete the Microsoft Learn modules recommended by the source assessment: "Secure a Microsoft Fabric data warehouse" (~74 min) for Maintain a data analytics solution, and "Ingest Data with Dataflows Gen2 in Microsoft Fabric" (~60 min) for Prepare data.
4. Practice hand-writing T-SQL aggregate queries (SUM vs. COUNT, MONTH()/YEAR() filters) and the visual query editor's Merge → Enable load → View query workflow, since both produced avoidable errors this round.
5. Retake a fresh practice assessment and track the same 4-domain breakdown again to confirm the gap is closing toward the 78%+ target.